Practical Preimages for 19- and 20-Step Reduced SHA-256: Exact Algebraic Padding Inversion, Context Shaping, and Causal Bit-Slice Pruning

Author: Sebastian Wain (`srw@egont.xyz`) **Date:** October 2026 **Repositories & Verification Suite:**

- 19-Step & 20-Step Padded Witnesses, Paper & Solvers: `https://github.com/srw/sha256-19step-preimages-with-padding`
- 20-Step Unpadded Compression Solver & Witness: `https://github.com/srw/sha256-20step-preimages-with-no-padding`
- Solver & Verification Suite: `solve20_padded_fast.c` (CPU), `solve20_padded_cuda.cu` & `solve20_padded_cuda_prod.cu` (CUDA GPU), `src/pre19.c` (19-Step Chosen-Target CPU), `solve20_zero_a3.c` (Zero-a_3 Unpadded), `test_all_12_theories.c` (14-Theory Algebraic Verification Suite), `demos_20step.txt`, `examples_abc_hello_20step.txt`, `witnesses_20step.txt`, `FOUND_20ROUND_PREIMAGE.txt`, `verify_all_witnesses.py`

Abstract

We present practical preimage attacks on the **19-step** and **20-step reduced SHA-256 hash functions** respecting the standard initial value (IV), Davies–Meyer feed-forward, and full FIPS 180-4 Merkle–Damgård padding [1] for arbitrary externally prescribed 256-bit target digests. Whereas prior demonstrated preimages at 18–20 steps primarily inverted the unpadded compression function [6, 7, 9] (with Basra [9] also reporting a 55-byte 20-step padded preimage for a structured target) and padding-aware meet-in-the-middle attacks at 24–47 steps remain strictly theoretical ($\geq 2^{240}$ evaluations) [2, 3, 4, 8], we construct valid 55-byte single-block and 119-byte two-block messages matching arbitrary prescribed 256-bit digests (including $\mathrm{SHA\text{-}256}(\text{“abc”})$, $\mathrm{SHA\text{-}256}(\text{“hello”})$, $\mathrm{SHA\text{-}256}(\text{“”})$, and 0^{256}). Building upon Basra's selector-saturation and backward padding-absorption framework [9], we introduce: (1) an exact $O(1)$ algebraic inversion of the low padding byte $(W_{13} \bmod 256) = \texttt{0x80}$ via 8 bit-disjoint pivot bits of $\Sigma_1(e_{13})$ that leaves (a_5, a_6) unconstrained by padding absorption so they can satisfy exact context shaping ($D_{\text{shape}} = a_5 \oplus a_6 = 0$) alongside Ch-annihilation; (2) a causal lower-triangular bit-slice equivalence theorem (**Carry-Sudoku**) that prunes 99.74% of right-hand candidate nodes across 7 causal stages (99.99% prior to evaluating $\Sigma_0(a_9)$); and (3) a closed-form $\mathbb{F}_2$ upper-half inverse paired with compact Q_0^{-1} tables (**Slot-Sudoku**) and 64-bit SWAR Level-3 filtering. For chosen-target 19-step padded preimages, our single-threaded solver `src/pre19.c` builds a 2^{29}-entry table in 16.8–19.1 s ($2 \times 2^{29} = 2^{30}$ Q_0 evaluations) and solves arbitrary 256-bit targets in 1.2–86.4 seconds on 1 CPU core (median 19.45 s, mean 30.2 s, $\approx 2^{26.7\text{–}26.9}$ context trials + $2^{20.8\text{–}26.8}$ chain starts, $2^{32.51\text{–}32.87}$ 19-step evals). For 20-step padded preimages, our host-verifiable CPU solver `solve20_padded_fast.c` streams $6{,}242.8$ schedule-consistent 19-step padded bridge candidates per second (`sol19/s` , 160.2 μs/sol19) on a 10-core Apple M4 CPU (16 threads), yielding an expected online time of 191.1 node-hours on 1 16-thread CPU node (23.9 node-hours across 8 CPU nodes, corresponding to $2^{46.56}$ same-host CPU-reference 20-step evaluations at 151.25 M evals/s); in supplementary GPU slice telemetry (`solve20_padded_cuda.cu` on $1\times$ NVIDIA L4), genuine-context streaming reaches $53{,}847.5$ sol19/s (22.16 GPU-h) on the 6.00 GiB table and $132{,}888.7$ sol19/s (8.98 GPU-h) with genuine Exact-T_{16} twin pairs. We also introduce a Zero-a_3 bi-directional bridge for the unpadded 20-step compression function (21.1 minutes observed / 33.1 minutes expected on 16 CPU cores, $2^{38.13}$ CPU-reference evaluations) and verify **46 distinct** 19- and 20-step witnesses (48 checks including benchmark overlap) across standard public-label and all-zero targets. These results concern step-reduced SHA-256 (19–20 of 64 steps) and do not threaten full 64-step SHA-256.

Keywords: SHA-256 · Preimage attack · Reduced-round cryptanalysis · FIPS 180-4 padding · Algebraic context shaping · Bit-slice pruning

---

1. Introduction

SHA-256 [1] hashes an L-bit message M by appending mandatory Merkle–Damgård strengthening (FIPS 180-4 §5.1.1)—a single `'1'` bit (`0x80` at a byte boundary), zero padding bits, and the 64-bit big-endian bit length L in the final two 32-bit words W_{14} and W_{15}—before applying a 64-step Davies–Meyer compression function initialized to the standard initial value IV.

In the cryptanalysis of step-reduced SHA-256, a persistent divide has separated **theoretical** attacks from **practical (demonstrated)** preimages:

- **Theoretical preimage and collision attacks:** Splice-and-cut meet-in-the-middle (MitM), partial-matching, biclique, and MILP-assisted attacks respect FIPS 180-4 padding from 24 steps [2] through 41–43 steps [3, 10], 45 steps [4], and 46–47 steps [8], with related differential and semi-free-start collision advances reaching 24–39 steps for SHA-256 (and 40 steps for SHA-224) [11, 12, 14, 15] building on the generalized characteristic framework introduced for SHA-1 in [13]. Unlike SHA-1, where full-round collisions have been practically computed [17, 18], all 24–47-step SHA-256 preimage attacks require between 2^{240} and $2^{255.6}$ compression evaluations and remain strictly theoretical.
- **Practical (demonstrated) preimages:** Algebraic and SAT-based encodings produced padded preimages up to 16 steps [5] and three mailing-list challenge messages at 17 steps (posted without an attack specification) [16]. More recently, SAT/Cube-and-Conquer [6, 7] produced verified preimages at 18 and 19 steps for the unpadded compression function (where all 16 message words $W_0, \ldots, W_{15}$ are unconstrained 32-bit degrees of

freedom), and Basra [9] introduced algebraic selector saturation and context shaping to solve 19- and 20-step unpadded compression targets as well as a 55-byte 20-step padded preimage for a structured target (`padded_r20` in [9], dated 17 October 2026).

For a single-block 55-byte ($L = 440$-bit) padded message, FIPS 180-4 [1] fixes 72 bits at the end of the message block:

$$W_{15} = 440\ (\texttt{0x000001b8}), \qquad W_{14} = 0\ (\texttt{0x00000000}), \qquad W_{13} \bmod 256 = \texttt{0x80}. \tag{E1}$$

In backward-and-forward inversion of steps $0..18$ (19-step) or $0..19$ (20-step), W_{13}, W_{14}, W_{15} are tightly coupled with the message schedule expansion $W_{16}, \ldots, W_{19}$ and the internal state window $(a_5, \ldots, a_{12})$. Satisfying all 72 padding bits simultaneously with the selector-saturation and majority-absorption conditions [9] that decouple the left and right halves of the compression function is the central barrier to inverting **arbitrary externally prescribed 256-bit targets** for the padded 19- and 20-step hash functions.

1.1 Upfront Demonstration of 19-Step and 20-Step Padded SHA-256 Preimages

To illustrate our results concretely before entering the algebraic derivations, below are four self-verifying **FIPS 180-4 padded preimages** for 20-step and 19-step SHA-256 ($\mathrm{SHA256}_{20}$ and $\mathrm{SHA256}_{19}$ with standard IV, Davies–Meyer feed-forward, and mandatory padding [1]; complete 46-witness verification suite in §8):

1. **20-Step Single-Block Padded Preimage for FIPS 180-4 Test Vector $H^* = \mathrm{SHA\text{-}256}(\text{“abc”})$:**

```
Target H*          = ba7816bf8f01cfea414140de5dae2223b00361a396177a9cb410ff61f20015ad
55-Byte Message M  = edf6e70a7001ae71b900f18119aaa1fdf6c1179b4268891262a9539
                     be04d41236685e0c102e0ad23165e8b7157669819302ddcb3c68226
Padded Block W0..7 = edf6e70a 7001ae71 b900f181 19aaa1fd f6c1179b 42688912 62a9539b e04d4123
Padded Block W8.15 = 6685e0c1 02e0ad23 165e8b71 57669819 302ddcb3 c6822680 00000000 000001b8
Verification       : SHA256_20(M) == H*  (W13 & 0xff = 0x80, W14 = 0, W15 = 440 bits)
```

2. **19-Step Single-Block Padded Preimages for $H^* = \mathrm{SHA\text{-}256}(\text{“abc”})$ and $H^* = 0000\ldots0000$:**

```
'abc' 55B Message  = 8ca254f8d04dd3fb04efaad6b1867588ffe833de49a144fedbf1839
                     422892dc9f5a9158d8fc063290a14a9a6c55dfa245642a0247ec45a
0^256 55B Message  = fc0c9d8eafb2fa329af316940c42a5eb9773cec39e4f7e396182628
                     7ae767638f92d4ce5fc5eb8cabd7606409364dda27a6c1bb5220246
Verification       : SHA256_19(M_abc) == SHA-256("abc") and SHA256_19(M_zero) == 0^256
```

3. **20-Step Single-Block Padded Preimage for All-Zero Digest $H^* = 0000\ldots0000$:**

```
Target H*          = 0000000000000000000000000000000000000000000000000000000000000000
55-Byte Message M  = ec531306c85ad4a4ad211d2e1a03ece4d0ad4a0e6b4c2b9436b3bab
                     dcbd520597196f52c042b8cd95f1a39f71b58b47b0647c78e65416f
Padded Block W8.15 = 7196f52c 042b8cd9 5f1a39f7 1b58b47b 0647c78e 65416f80 00000000 000001b8
Verification       : SHA256_20(M) == 0^256  (W13 & 0xff = 0x80, W14 = 0, W15 = 440 bits)
```

4. **20-Step & 19-Step Two-Block (119-Byte, Plain-ASCII Block 1) Padded Preimages for $H^* = \mathrm{SHA\text{-}256}(\text{“”})$:**

```
Target H*          = e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855
Block 1 (64B ASCII)= "This first block is plain text; the second one was computed....."
20-Step Block 2 M2 = df28e21bb6b61202ba87baa72096ee0a3ac325cf1f12056e7cfe2ae
                     1735b4d106b8f6e0b22eb9b504ed9368991944b0a115a934f17c2c4
19-Step Block 2 M2 = fe409cc930128a720481f86c0503d7a018519a129df449139720bb5
                     eef520895f8b515685bb404817be301880f0caabe02fc9e5f1c95c0
Padding Suffix     : ...80 00000000 000003b8 (L = 952 bits = 119 bytes; full Merkle-Damgard 2-block chaining)
```

Table 1.1: Comparison of theoretical and practical (demonstrated) preimage attacks on step-reduced SHA-256.

Work & Reference	Steps	Attacked Object	FIPS 180-4 Padding?	Target Type	Complexity / Runtime	Demonstrated Witness?
Isobe & Shibutani [2]	24	Hash (1 block)	Yes	Arbitrary	2^{240} evaluations	No (theoretical)
Sasaki, Wang & Aoki [10]	41	Hash	Yes	Arbitrary	$2^{253.5}$ evaluations	No (theoretical)
Aoki et al. [3]	43	Hash	Yes	Arbitrary	$2^{254.9}$ evaluations	No (theoretical)

Work & Reference	Steps	Attacked Object	FIPS 180-4 Padding?	Target Type	Complexity / Runtime	Demonstrated Witness?
Khovratovich et al. [4]	45 / 52	Hash / Compression (pseudo)	Yes / No	Arbitrary	$2^{255.5}/2^{255.0}$ evaluations	No (theoretical)
Guo et al. [8]	46 / 47	Hash	Yes	Arbitrary	$2^{254.5}/2^{255.6}$ evaluations	No (theoretical)
Homsirikamol et al. [5]	16	Hash (446-bit)	Yes	Fixed	≈ 1 h (Table 1 SAT)	No (not printed)
McDair [16]	17	Hash (31B / 55B)	Yes	1 challenge	Unspecified (no method)	Yes (3 messages)
Davydov et al. [6]	17 / 18	Compression	No	All-zero (`00..00`)	Seconds (SAT)	Yes (unpadded only)
Zaikin [7]	19	Compression	No	All-ones (`ff..ff`)	18 h 33 min on 192 cores (SAT)	Yes (unpadded only)
Basra [9]	19 / 20	Compression & Structured Padded	No (19–20) / Yes (20 structured)	Arbitrary (unpadded) / Structured (padded 20)	~ 25 s/ctx on H100 (19-step); 20-step unpadded & structured padded	Yes (unpadded & 1 structured padded 20-step)
This Work (`solve20_zero_a3.c`)	**20**	**Compression**	**No**	**`deadbeef...` + 12 Labels**	**33.1 min expected / `21.1 min` observed (16 CPU cores; $2^{38.13}$ CPU-ref)**	**Yes (13 witnesses, §8.4)**
This Work (`src/pre19.c` & Exact $D = 0$)	**19**	**Full Hash (55B / 119B)**	**Yes [1]**	**`00..00` + 14 Labels + `""`**	**`1.2–86.4 s` 1-core search (median `19.45 s`, $2^{32.51–32.87}$ 19-step evals)**	**Yes (16 witnesses, §8.2–§8.4)**
This Work (`solve20_padded_fast.c` / `cuda.cu`)	**20**	**Full Hash (55B / 119B)**	**Yes [1]**	**`00..00` + 14 Labels + `""`**	**`191.1 node-h` 1× CPU ($2^{46.56}$ CPU-ref); *supp. GPU:* `8.98–22.16 h` 1× L4**	**Yes (16 witnesses, §8.1–§8.4)**

*(Note on Table 1.1: In Khovratovich et al. [4], 45 steps is a full hash preimage and 52 steps is a compression pseudo-preimage; in Basra [9], `padded_r20` solves a structured digest [`c799d851...`] constructed with $e_9 = e_{10} = e_{15} =$ `0xffffffff` and $a_5 = a_6 = a_{10}$. For our solvers, we distinguish a **chosen-target 19-step padded preimage**—solving a prescribed 19-step digest H_{19}^* where $a_{12} \dots a_{19}$ are fixed, taking 1.2–86.4 s of single-core search time in `src/pre19.c` after a 16.8–19.1 s one-time 2^{29}-entry table build—from a **19-step bridge candidate (`sol19`)** generated at $6{,}242.8$ sol19/s on a 10-core CPU node in `solve20_padded_fast.c` [or $53{,}847.5$–$132{,}888.7$ sol19/s in supplementary 1× L4 GPU slice telemetry in `solve20_padded_cuda.cu`] during a 20-step search where a_{12} is free. Elapsed-time proxy `CPU-ref` multiplies wall-clock seconds by the measured same-host 10-core CPU 20-step SHA-256 reference rate of 151.25 M evals/s, where 1 node-h $= 2^{38.99}$ CPU-ref; see §6.5.)*

1.2 Technical Contributions and Organization

1. **Closed-Form Pivot Inversion of FIPS 180-4 Padding Compatible with Exact Context Shaping (§3–§4, `[E15]` – `[E22]`):** Basra [9] showed that the FIPS 180-4 padding words $(W_{15} = 440, W_{14} = 0, W_{13} \bmod 256 =$ `0x80`$)$ can be absorbed backward into (a_7, a_6, a_5) at the cost of three context words and demonstrated a 55-byte 20-step padded witness for a structured target (`padded_r20` in [9]). For **arbitrary externally prescribed 256-bit targets**, we show how to absorb $W_{15} = 440$ and $W_{14} = 0$ into (a_8, a_7) while inverting $(W_{13} \bmod 256) =$ `0x80` directly via 8 bit-disjoint pivot bits (`{18, 25..31}`) of $\Sigma_1(e_{13})$ with 100% inner-loop validity—leaving (a_5, a_6) unconstrained by padding absorption so they can satisfy exact shaping $D_{\text{shape}} = a_5 \oplus a_6 = 0$ alongside Ch-annihilation ($e_{15} =$ `0xffffffff`, $a_{10} = a_{11}$).
2. **Causal Bit-Slice Right-Context Pruning ("Carry-Sudoku", §5, `[E23]` – `[E25]`):** We prove **Theorem 1** (`[E23]`): $a_5 = a_6$ decomposes into a lower-triangular carry condition $((S \oplus X) \wedge D) = 0$ (where $D = a_8 \oplus a_7$ is the bitwise XOR mask) that is completely independent of a_6 and $\Sigma_0(a_9)$, and a complementary condition $((a_6 \oplus S) \wedge \neg D) = 0$. Pruning e_{13} across 7 causal bit-slices reduces the 2^{24} candidate space to an average of $43{,}580$ node evaluations per a_{12} block ($385\times$ / 99.74% node reduction across all 7 stages, evaluating $\Sigma_0(a_9)$ only on the $\approx 1{,}660$–$2{,}149$ full-carry survivors per block, i.e., 99.99% pre-$\Sigma_0(a_9)$ pruning) with 100% completeness across each 2^{24}-candidate a_{12} block (and the estimated $U \approx 1.68 \times 10^7 \approx 2^{24.0}$ exact $D_{\text{shape}} = 0$ contexts per 20-step target).
3. **Closed-Form $\mathbb{F}_2$ Reconstruction, Compact Q_0^{-1} Tables ("Slot-Sudoku"), and T_{16} Twin Coalescing (§6, `[E27]` – `[E37]` , `[E42]` – `[E45]`):** We prove **Theorem 2** (`[E28]` – `[E29]`), recovering the upper 16 bits of $u \in Q_0^{-1}(y)$ in closed form in registers with zero lookup tables. Pairing this with compact Q_0^{-1} tables achieves $6{,}242.8$ sol19/s on our host-verifiable 10-core CPU solver (`solve20_padded_fast.c`), and $53{,}847.5$–$132{,}888.7$ sol19/s on genuine contexts in supplementary 1× NVIDIA L4 GPU slice telemetry (`solve20_padded_cuda.cu`).
4. **Zero-a_3 Bi-Directional Bridge, Complete 46-Witness Suite, and Construction-Specific Bottleneck Analysis (§7–§9, `[E38]` – `[E41]`):** We present a Zero-a_3 bridge for unpadded 20-step compression (§7), provide the complete, un-truncated hex witnesses for all **46 distinct** 19-step and 20-step single-block padded, two-block chosen-ASCII-prefix padded, unpadded, and regression benchmarks (§8, 48 checks including `"abc"` demo/benchmark overlap), and analyze 15 explored sub-2^{32} algebraic and statistical directions characterizing the degree-of-freedom conservation laws of the Carry-Sudoku / Slot-Sudoku architecture (§9.2).

2. Preliminaries and the Scalar a-Sequence Formulation

All arithmetic is modulo 2^{32}. Denote 32-bit right rotation by $\mathrm{ROR}(x, n)$ and logical right shift by $\mathrm{SHR}(x, n)$. The SHA-256 boolean and $\mathbb{F}_2$-linear functions [1] are:

$$\begin{aligned} \mathrm{Ch}(e,f,g) &= (e \wedge f) \oplus (\neg e \wedge g), & \mathrm{Maj}(a,b,c) &= (a \wedge b) \oplus (a \wedge c) \oplus (b \wedge c), \\ \Sigma_0(x) &= \mathrm{ROR}(x,2) \oplus \mathrm{ROR}(x,13) \oplus \mathrm{ROR}(x,22), & \Sigma_1(x) &= \mathrm{ROR}(x,6) \oplus \mathrm{ROR}(x,11) \oplus \mathrm{ROR}(x,25), \\ \sigma_0(x) &= \mathrm{ROR}(x,7) \oplus \mathrm{ROR}(x,18) \oplus \mathrm{SHR}(x,3), & \sigma_1(x) &= \mathrm{ROR}(x,17) \oplus \mathrm{ROR}(x,19) \oplus \mathrm{SHR}(x,10). \end{aligned} \tag{E2}$$

For step indices $t \geq 16$, the message schedule expands $W_0, \ldots, W_{15}$ via:

$$W_t = \sigma_1(W_{t-2}) + W_{t-7} + \sigma_0(W_{t-15}) + W_{t-16}. \tag{E3}$$

Following [9], we define the coupled schedule operator $Q_0 : \mathbb{Z}_{2^{32}} \to \mathbb{Z}_{2^{32}}$ by:

$$Q_0(u) = \sigma_0(u) - u \pmod{2^{32}}. \tag{E4}$$

2.1 Reduction to the Scalar a-Sequence (`[E5]` – `[E7]`)

Let $(a_t, b_t, c_t, d_t, e_t, f_t, g_t, h_t)$ be the 8 working registers after step $t-1$ (with $t = 0$ denoting IV). Because each step shifts $b \leftarrow a, c \leftarrow b, d \leftarrow c$ and $f \leftarrow e, g \leftarrow f, h \leftarrow g$, the entire 8-register trajectory across R steps is uniquely parameterized by a single scalar sequence $a_{-7}, a_{-6}, \ldots, a_R$ [3, 9]:

- **Initial Boundary ($s \in \{-7, \ldots, 0\}$):** Fixed uniquely by IV (`window_from_state` in `solve20_padded_fast.c`):

$$(a_0, a_{-1}, a_{-2}, a_{-3}) = (\mathrm{IV}_0, \mathrm{IV}_1, \mathrm{IV}_2, \mathrm{IV}_3), \qquad (e_0, e_{-1}, e_{-2}, e_{-3}) = (\mathrm{IV}_4, \mathrm{IV}_5, \mathrm{IV}_6, \mathrm{IV}_7). \tag{E5}$$

- **Terminal Boundary ($s \in \{R-7, \ldots, R\}$):** For an R-step target digest $H^* = (H_0^*, \ldots, H_7^*)$, Davies–Meyer feed-forward fixes the terminal state $st_i = H_i^* - \mathrm{IV}_i$, which uniquely determines the 8 terminal words $a_{R-7}, \ldots, a_R$:

$$\begin{aligned} &\text{For } R = 19: \quad a_{12}, a_{13}, \ldots, a_{19} \text{ are fixed by } H^* - \mathrm{IV}, \\ &\text{For } R = 20: \quad a_{13}, a_{14}, \ldots, a_{20} \text{ are fixed by } H^* - \mathrm{IV}. \end{aligned} \tag{E6}$$

- **Internal State & Message Recovery:** For every step index s and $t \in \{0, \ldots, R-1\}$, the e-register e_s is an exact function of five consecutive a-words ($a_{s-4}, \ldots, a_s$), and the message word W_t is an exact function of a_{t-3} together with five e-words ($e_{t-3}, \ldots, e_{t+1}$, spanning nine consecutive a-words $a_{t-7}, \ldots, a_{t+1}$ when expanded; `E_arr` and `Wt_arr` in `solve20_padded_fast.c`):

$$\begin{aligned} e_s &= a_s - \Sigma_0(a_{s-1}) - \mathrm{Maj}(a_{s-1}, a_{s-2}, a_{s-3}) + a_{s-4}, \\ W_t &= e_{t+1} - a_{t-3} - e_{t-3} - \Sigma_1(e_t) - \mathrm{Ch}(e_t, e_{t-1}, e_{t-2}) - K_t. \end{aligned} \tag{E7}$$

Thus, for $R = 20$ (resp. $R = 19$), the only free internal state variables between the IV boundary and the target boundary are the **12 words** $a_1, a_2, \ldots, a_{12}$ (resp. **11 words** $a_1, \ldots, a_{11}$). Any assignment to these words automatically hashes to H^* after R steps if and only if the recovered message words $W_0, \ldots, W_{R-1}$ from `[E7]` satisfy the message schedule equations `[E3]` for $t = 16, \ldots, R-1$ and the FIPS 180-4 padding constraints `[E1]` .

---

3. Selector Saturation and Exact Context Shaping ($D_{\text{shape}} = 0$)

In general, $W_{16}, W_{17}, W_{18}, W_{19}$ depend on the Left variables (a_1, a_2, a_3, a_4) both through the early message words $(W_0, W_1, W_2, W_3, W_4)$ on the right-hand side of `[E3]` and through $(W_9, W_{10}, W_{11}, W_{12}, W_{14}, W_{15})$ via the state words e_5, e_6, e_7, e_8.

Following Basra's selector-saturation principle [9], we pin the two middle selector registers:

$$e_9 = \texttt{0xffffffff}, \qquad e_{10} = \texttt{0xffffffff}. \tag{E8}$$

Because $\mathrm{Ch}(\texttt{0xffffffff}, y, z) = y$, setting $e_9 = e_{10} = \texttt{0xffffffff}$ blocks (e_7, e_8) inside $\mathrm{Ch}(e_9, e_8, e_7) = e_8$, $\mathrm{Ch}(e_{10}, e_9, e_8) = \texttt{0xffffffff}$, and $\mathrm{Ch}(e_{11}, e_{10}, e_9) = \texttt{0xffffffff}$. Expanding `[E7]` at $s = 9, 10$ expresses a_5 and a_6 purely in terms of the Right words (a_7, a_8, a_9, a_{10}):

$$
\begin{aligned}
a_5 &= \texttt{0xffffffff} - a_9 + \Sigma_0(a_8) + \mathrm{Maj}(a_8, a_7, a_6), \\
a_6 &= \texttt{0xffffffff} - a_{10} + \Sigma_0(a_9) + \mathrm{Maj}(a_9, a_8, a_7).
\end{aligned}
\tag{E9}
$$

3.1 Exact Shaping ($D_{\text{shape}} = a_5 \oplus a_6 = 0$) vs. Chosen-Target 19-Step Shaping (`[E10]` – `[E14]`)

Let $D_{\text{shape}} := a_5 \oplus a_6$ denote the bitwise XOR difference between a_5 and a_6 (note that $D_{\text{shape}} = 0 \iff a_5 = a_6 \iff a_6 - a_5 \equiv 0 \pmod{2^{32}}$). Under `[E8]` ($e_9 = e_{10} = \texttt{0xffffffff}$), $\mathrm{Ch}(e_9, e_8, e_7) = e_8$ and $\mathrm{Ch}(e_{10}, e_9, e_8) = \mathrm{Ch}(e_{11}, e_{10}, e_9) = \texttt{0xffffffff}$. Expanding `[E7]` at $t = 9, 10, 11, 12$ isolates how $W_9, W_{10}, W_{11}, W_{12}$ depend on the Left variables (a_1, a_2, a_3, a_4) [9]:

$$
\begin{aligned}
W_9 &= c_9 - a_2 - a_4 + \mathrm{Maj}(a_5, a_4, a_3), \\
W_{10} &= c_{10} - a_3 + \big(\mathrm{Maj}(a_6, a_5, a_4) - (a_6 \wedge a_5)\big), \\
W_{11} &= c_{11} - a_4, \\
W_{12} &= c_{12} = \mathrm{const}(\mathrm{Right}),
\end{aligned}
\tag{E10}
$$

where $c_9, c_{10}, c_{11}, c_{12}$ depend **only** on the Right context $(a_5, \ldots, a_{13})$.

When we enforce **exact context shaping**:

$$
D_{\text{shape}} = a_5 \oplus a_6 = 0 \iff a_5 = a_6,
\tag{E11}
$$

the majority term in W_{10} becomes identically constant for **every** $a_4 \in \mathbb{Z}_{2^{32}}$ because $\mathrm{Maj}(a_6, a_5, a_4) = \mathrm{Maj}(a_5, a_5, a_4) = a_5 = a_6 \wedge a_5$, yielding $W_{10} = c_{10} - a_3$ with zero a_4 leakage. *(For a **chosen 19-step target** H^*_{19}, where $a_{12} \ldots a_{19}$ are all fixed by `[E6]`, we leave $a_{10} \in [0, 2^{32})$ free instead of setting $a_{10} = a_{11}$, since Step 20 is absent and $e_{15} = \texttt{0xffffffff}$ is not required. With free a_{10}, one can either enforce exact shaping $D_{\text{shape}} = 0$—which produced our 19-step 0^{256} witness in §8.2—or use soft shaping $\mathrm{popcount}(a_5 \oplus a_6) \le 3$ as in `src/pre19.c`, which enlarges the context pool by $5{,}489\times$ and generated the 14 public-label 19-step witnesses in Table 8.3 in 1.2–86.4 s of single-core CPU search time per target after a one-time 16.8–19.1 s table build.)*

Simultaneously, whenever the remaining nonlinear condition in W_9 (`[E10]`) holds, the middle schedule words reduce to exact affine forms in (a_2, a_3, a_4) while $W_{12}, W_{13}, \ldots, W_{19}$ are **100% independent of** (a_1, a_2, a_3, a_4):

$$
W_9 = c_9 - a_2, \qquad W_{10} = c_{10} - a_3, \qquad W_{11} = c_{11} - a_4, \qquad W_{12} = c_{12},
\tag{E12}
$$

where $-a_4$ inside e_8 and $+\mathrm{Maj}(a_5, a_4, a_3) = +a_4$ inside e_6 cancel identically in W_9 under the Level-3 majority-absorption condition:

$$
\mathrm{Maj}(a_5, a_4, a_3) = a_4 \iff \big((a_4 \oplus a_5) \wedge M_3\big) = 0, \qquad M_3 := \neg(a_3 \oplus a_5).
\tag{E13}
$$

Table 3.1 summarizes the resulting bit-level state conditions across all 20 steps in the format of [3, 8, 11].

Table 3.1: Bit-level state conditions, neutral/coupled words, and schedule constraints across steps $s \in \{-7, \ldots, 20\}$.

Step Range s	Register a_s Condition	Register e_s Condition	Message Word W_{s-1} Role & Constraint	Governing Equation
$-7 \ldots 0$	Fixed by $\mathrm{IV}_{0..3}$	Fixed by $\mathrm{IV}_{4..7}$	— (Initial boundary)	`[E5]`
1	Free root $a_1 \in \mathbb{Z}_{2^{32}}$	$e_1(a_1)$	$W_0 = a_1 + C_{W_0}$ (Root of 3-level Left tree)	`[E7]` , `[E14]`
$2, 3$	$a_2 = W_1 + G_1$, $a_3 = W_2 + G_2$	$e_2(a_{1..2})$, $e_3(a_{1..3})$	$W_1 \in Q_0^{-1}(T_{16} + F_1)$, $W_2 \in Q_0^{-1}(T_{17} + G_2 - W_1)$; $\mathrm{wt}(M_3) \le 14$	`[E14]` , `[E34]`
4	$\big((a_4 \oplus a_5) \wedge \neg(a_3 \oplus a_5)\big) = 0$	$e_4(a_{1..4})$	$W_3 \in Q_0^{-1}(T_{18} + G_3 - W_2)$ ($\mathrm{Maj}(a_5, a_4, a_3) = a_4$)	`[E13]` , `[E14]`
$5, 6$	**Exact Shaping:** $a_5 = a_6$ ($D = 0$)	e_5, e_6 (coupled)	W_4 tested at step 20 via $\sigma_0(W_4) + W_3 = T_{19}$; W_5 free	`[E11]` , `[E36]`
$7, 8$	Closed-form $a_7(e_{13})$, $a_8 = \mathrm{const}$	e_7, e_8 (absorbed)	W_6, W_7 free; $-a_4$ in e_8 cancels $+\mathrm{Maj}$ in e_6 inside W_9	`[E17]` , `[E22]`
$9, 10$	$a_9 = e_{13} - c_{e_{13}}$, $a_{10} = a_{11}$	$e_9 = e_{10} = \texttt{0xffffffff}$	W_8 free; $W_9 = c_9 - a_2$ (under `[E13]`)	`[E8]` , `[E12]`
$11, 12$	a_{11} fixed by e_{15}, a_{12} seed	$e_{11}(e_{13})$, $e_{12} = \mathrm{const}$	$W_{10} = c_{10} - a_3$, $W_{11} = c_{11} - a_4$ (zero a_4 leakage into W_{10})	`[E12]` , `[E16]`
$13, 14$	a_{13}, a_{14} fixed by $H^* - \mathrm{IV}$	e_{13} (7-stage causal), e_{14}	$W_{12} = c_{12}$; **Padding:** $W_{13} \bmod 256 = \texttt{0x80}$	`[E12]` , `[E21]`

Step Range s	Register a_s Condition	Register e_s Condition	Message Word W_{s-1} Role & Constraint	Governing Equation
$15, 16$	a_{15}, a_{16} fixed by $H^* - \mathrm{IV}$	$e_{15} = \texttt{0xffffffff}$, e_{16}	**Padding:** $W_{14} = \texttt{0x00000000}$, $W_{15} = \texttt{0x000001b8}$ (440)	`[E1]`, `[E22]`
$17 \dots 20$	$a_{17..20}$ fixed by $H^* - \mathrm{IV}$	$e_{17..20}$ fixed	Schedule words $W_{16..19}$ define $(T_{16}, T_{17}, T_{18}, T_{19})$	`[E6]`, `[E14]`

For $t \in \{1, 2, 3\}$, rearrangement of `[E7]` writes $a_{t+1} = W_t + G_t(a_1, \dots, a_t)$, where $G_t(a_1, \dots, a_t) := \Sigma_0(a_t) + \mathrm{Maj}(a_t, a_{t-1}, a_{t-2}) + e_{t-3} + \Sigma_1(e_t) + \mathrm{Ch}(e_t, e_{t-1}, e_{t-2}) + K_t$. Setting $F_1(a_1) := G_1(a_1) - W_0(a_1)$ and substituting $a_{t+1} = W_t + G_t$ together with `[E12]` ($W_9 = c_9 - a_2$, $W_{10} = c_{10} - a_3$, $W_{11} = c_{11} - a_4$) into the message schedule equations `[E3]` for $t = 16, 17, 18$ rewrites them as three chained Q_0-inversions:

$$\begin{aligned}
&\text{Level 1 } (t = 16): \quad Q_0(W_1) = T_{16} + F_1(a_1), && T_{16} := W_{16} - \sigma_1(W_{14}) - c_9, \\
&\text{Level 2 } (t = 17): \quad Q_0(W_2) = T_{17} + G_2(a_1, a_2) - W_1, && T_{17} := W_{17} - \sigma_1(W_{15}) - c_{10}, \\
&\text{Level 3 } (t = 18): \quad Q_0(W_3) = T_{18} + G_3(a_1, a_2, a_3) - W_2, && T_{18} := W_{18} - \sigma_1(W_{16}) - c_{11}.
\end{aligned} \tag{E14}$$

---

4. Closed-Form Pivot Inversion of FIPS 180-4 Padding (`W13..W15`)

To construct a valid 55-byte single-block padded message (`[E1]`), the Right variables $(a_5, \dots, a_{12})$ must simultaneously satisfy $W_{15} = 440$, $W_{14} = 0$, $(W_{13} \bmod 256) = \texttt{0x80}$, $e_9 = e_{10} = \texttt{0xffffffff}$ (`[E8]`), and $a_5 = a_6$ (`[E11]`). Basra [9] noted that $(W_{15}, W_{14}, W_{13} \bmod 256)$ can be absorbed backward into (a_7, a_6, a_5) at the cost of three context words; below we show how to absorb $W_{15} = 440$ and $W_{14} = 0$ into (a_8, a_7) while inverting $(W_{13} \bmod 256) = \texttt{0x80}$ directly via 8 bit-disjoint pivot bits of $\Sigma_1(e_{13})$, leaving (a_5, a_6) unconstrained by padding absorption so they can satisfy exact shaping $a_5 = a_6$ (`[E23]`).

4.1 Closed-Form Absorption of $W_{15} = 440$ and $W_{14} = 0$ (`[E15]`–`[E17]`)

Given (a_{10}, a_{11}, a_{12}) (for $R = 20$, where $a_{13..20}$ are fixed by H^*), the registers e_{16}, e_{15}, e_{14} are determined via `[E7]`:

$$\begin{aligned}
e_{16} &= a_{16} - \Sigma_0(a_{15}) - \mathrm{Maj}(a_{15}, a_{14}, a_{13}) + a_{12}, \\
e_{15} &= a_{15} - \Sigma_0(a_{14}) - \mathrm{Maj}(a_{14}, a_{13}, a_{12}) + a_{11}, \\
e_{14} &= a_{14} - \Sigma_0(a_{13}) - \mathrm{Maj}(a_{13}, a_{12}, a_{11}) + a_{10}.
\end{aligned} \tag{E15}$$

Expanding $W_{15} = 440$ and $W_{14} = 0$ via `[E7]` isolates e_{12} and e_{11} as closed-form functions of a single 32-bit word e_{13} (equivalently $a_9 = e_{13} - c_{e_{13}}$):

$$\begin{aligned}
e_{12} &= \underbrace{(e_{16} - a_{12} - \Sigma_1(e_{15}) - K_{15} - 440)}_{c_{e_{12}}} - \mathrm{Ch}(e_{15}, e_{14}, e_{13}), \\
e_{11} &= \underbrace{(e_{15} - a_{11} - \Sigma_1(e_{14}) - K_{14})}_{c_{e_{11}}} - \mathrm{Ch}(e_{14}, e_{13}, e_{12}).
\end{aligned} \tag{E16}$$

Then a_8 and a_7 are recovered in closed form:

$$a_8 = e_{12} + \Sigma_0(a_{11}) - a_{12} + \mathrm{Maj}(a_{11}, a_{10}, a_9), \qquad a_7 = e_{11} + \Sigma_0(a_{10}) - a_{11} + \mathrm{Maj}(a_{10}, a_9, a_8). \tag{E17}$$

Thus **every** choice of $(a_{12}, a_{11}, a_{10}, e_{13})$ satisfies $W_{15} = 440$ and $W_{14} = 0$ identically.

4.2 Closed-Form Pivot Assignment for $(W_{13} \bmod 256) = \texttt{0x80}$ (`[E18]`–`[E22]`)

Since $e_{10} = \texttt{0xffffffff}$, the formula for W_{13} (`[E7]`) is:

$$W_{13} = \underbrace{(e_{14} - a_{10} - \texttt{0xffffffff} - K_{13})}_{c_{W_{13}}} - \Sigma_1(e_{13}) - \mathrm{Ch}(e_{13}, e_{12}, e_{11}). \tag{E18}$$

Reducing `[E18]` modulo 256:

1. Because Ch is bitwise and modular subtraction propagates carries only from lower bits to higher bits, the low 8 bits $e_{12,\mathrm{lo}} := e_{12} \bmod 256$ and $e_{11,\mathrm{lo}} := e_{11} \bmod 256$ in `[E16]` depend **only on the low 8 bits** $e_{13,\mathrm{lo}} := e_{13} \bmod 256$.
2. Once $e_{13,\mathrm{lo}} \in [0, 255]$ is chosen, the required low byte of $\Sigma_1(e_{13})$ is fixed:

$$R_{\mathrm{lo}} := \left(c_{W_{13}} - \mathrm{Ch}(e_{13,\mathrm{lo}}, e_{12,\mathrm{lo}}, e_{11,\mathrm{lo}}) - \texttt{0x80}\right) \bmod 256. \tag{E19}$$

3. Inspecting bit $b \in \{0,\dots,7\}$ of $\Sigma_1(e_{13}) = \mathrm{ROR}(e_{13},6) \oplus \mathrm{ROR}(e_{13},11) \oplus \mathrm{ROR}(e_{13},25)$:

$$\big(\Sigma_1(e_{13})\big)_b = (e_{13})_{b+6} \oplus (e_{13})_{b+11} \oplus (e_{13})_{(b+25) \bmod 32}. \tag{E20}$$

- For $b \in \{0,\dots,6\}$, $(b+25) \bmod 32 = b+25 \in \{25,\dots,31\}$, which appears **nowhere else** in `[E20]`.
- For $b = 7$, $(7+25) \bmod 32 = 0 \in \{0,\dots,7\}$, while $(e_{13})_{7+11} = (e_{13})_{18}$ appears **nowhere else** in `[E20]`.
- Furthermore, bits $19..24$ of e_{13} **do not appear anywhere in `[E20]`** and **do not affect** $e_{13,\mathrm{lo}}$.

Therefore, for any assignment to bits $0..17$ of e_{13} ($e_{13,\mathrm{base}}$), the 8 pivot bits (`bit 18` and `bits 25..31`) are uniquely determined in closed form:

$$\Delta := \big(R_{\mathrm{lo}} \oplus \Sigma_1(e_{13,\mathrm{base}})\big) \bmod 256, \qquad e_{13,\mathrm{fixed}} := e_{13,\mathrm{base}} \mid \big((\Delta \wedge \texttt{0x7f}) \ll 25\big) \mid \big(((\Delta \gg 7) \wedge 1) \ll 18\big), \tag{E21}$$

while bits $19..24$ remain 6 completely free degrees of freedom where 100% **of assignments satisfy** $(W_{13} \bmod 256) = \texttt{0x80}$.

Finally, in `solve20_padded_fast.c` and `solve20_padded_cuda.cu`, we apply Ch**-annihilation** by pinning $e_{15} = \texttt{0xffffffff}$ (fixing a_{11} from a_{12} via `[E15]`) and setting $a_{10} = a_{11}$. This simultaneously forces $\mathrm{Ch}(e_{15},e_{14},e_{13}) = e_{14}$ (**making e_{12} and a_8 100% constant across all 2^{24} choices of e_{13}**), simplifies $\mathrm{Maj}(a_{11},a_{10},a_9) = a_{11}$, and linearizes $\mathrm{Maj}(a_{10},a_9,a_8) = (a_{10} \wedge a_8) + ((a_{10} \oplus a_8) \wedge a_9)$:

$$e_{15} = \texttt{0xffffffff}, \quad a_{10} = a_{11} \implies e_{12} = c_{e_{12}} - e_{14} = \mathrm{const}, \quad a_8 = e_{12} + \Sigma_0(a_{11}) - a_{12} + a_{11} = \mathrm{const}. \tag{E22}$$

---

5. Carry-Sudoku: 7-Stage Causal Bit-Slice Right-Context Search

Even with `[E21]` – `[E22]`, each a_{12} defines 2^{24} padded candidates for e_{13}, and testing $a_5 = a_6$ (`[E11]`) naively requires evaluating $\Sigma_0(a_9)$ and a_6 across those candidates. We now prove that causal lower-triangular bit-slice pruning—which we term **Carry-Sudoku**—reduces the 2^{24} candidate tree to an average of $43{,}580$ node evaluations (99.74% pruning, or $385\times$ fewer evaluations) *before* $\Sigma_0(a_9)$ or a_6 is ever computed.

5.1 The Exact Equivalence Theorem (`[E23]`)

Theorem 1 (Carry-Sudoku Exact Equivalence). *Under `[E9]`, define $X := \texttt{0xffffffff} - a_9 + \Sigma_0(a_8)$, the bitwise XOR mask $D := a_8 \oplus a_7$, and $S := X + \mathrm{Ch}(D,X,a_8) \pmod{2^{32}}$. Then:*

$$a_5 = a_6 \iff \Big(\big((S \oplus X) \wedge D\big) = 0\Big) \ \wedge \ \Big(\big((a_6 \oplus S) \wedge \neg D\big) = 0\Big). \tag{E23}$$

Proof. By `[E9]`, $a_5 = X + \mathrm{Maj}(a_8,a_7,a_6)$. Decompose $\mathrm{Maj}(a_8,a_7,a_6)$ bitwise with respect to $D = a_8 \oplus a_7$:

- On bit positions where $D_b = 0$ ($a_{8,b} = a_{7,b}$), $\mathrm{Maj}(a_8,a_7,a_6)_b = a_{8,b}$ is independent of a_6.
- On bit positions where $D_b = 1$ ($a_{8,b} \neq a_{7,b}$), $\mathrm{Maj}(a_8,a_7,a_6)_b = a_{6,b}$.

Hence $\mathrm{Maj}(a_8,a_7,a_6) = (D \wedge a_6) \oplus (\neg D \wedge a_8) = \mathrm{Ch}(D,a_6,a_8)$. Now compare $a_5 = X + \mathrm{Ch}(D,a_6,a_8)$ with $S = X + \mathrm{Ch}(D,X,a_8)$ by induction from bit $b = 0$ to 31:

- Suppose $a_5 = a_6$ on bits $0..b-1$, so the incoming carry c_b into bit b of $X + \mathrm{Ch}(D,a_6,a_8)$ equals the incoming carry into bit b of $S = X + \mathrm{Ch}(D,X,a_8)$:
- If $D_b = 1$, then $\mathrm{Ch}(D,a_6,a_8)_b = a_{6,b}$, so $a_{5,b} = X_b \oplus a_{6,b} \oplus c_b$. Thus $a_{5,b} = a_{6,b} \iff X_b \oplus c_b = 0$, and when $X_b \oplus c_b = 0$, the outgoing carry from bit b is $\mathrm{Maj}(X_b,a_{6,b},c_b) = X_b = \mathrm{Maj}(X_b,X_b,c_b)$, which is **identical** to the outgoing carry of $S = X + \mathrm{Ch}(D,X,a_8)$ where $S_b = X_b \oplus X_b \oplus c_b = c_b = X_b$, i.e. $(S_b \oplus X_b) = 0$.
- If $D_b = 0$, then $\mathrm{Ch}(D,a_6,a_8)_b = a_{8,b} = \mathrm{Ch}(D,X,a_8)_b$, so $a_{5,b} = S_b$ and the outgoing carry is again identical to that of S. Thus $a_{5,b} = a_{6,b} \iff a_{6,b} = S_b$. $\square$

5.2 7-Stage Causal Bit-Slice Pruning (`[E24]` – `[E26]`)

Theorem 1 has a decisive algorithmic consequence: the first condition in `[E23]`,

$$\big((S \oplus X) \wedge D\big) = 0, \qquad S = X + \mathrm{Ch}(D,X,a_8), \tag{E24}$$

does not depend on a_6 or $\Sigma_0(a_9)$ at all, and because $a_9 = e_{13} - c_{e_{13}}$, $X = c_{a_5} - a_9$, and $a_7 = c_{a_7,\mathrm{full}} - (e_{14} \wedge e_{13}) + ((a_{10} \oplus a_8) \wedge a_9)$ involve only bitwise operations and modular additions/subtractions, **the lowest k bits of X, a_7, D, and S depend exclusively on the lowest k bits of e_{13}**.

Consequently, `ctx_worker` (`solve20_padded_fast.c`) on CPU and `ctx_gen_cuda_kernel` / `ctx_gen_fast_cuda_kernel` (`solve20_padded_cuda.cu`, `solve20_padded_cuda_prod.cu`) on GPU assign e_{13} from LSB to MSB in **7 exact causal stages** (formalized in **Algorithm 1**):

$$
\begin{aligned}
\text{Stage 1a (bits 0..3):}\quad & \big((S \oplus X) \wedge D \wedge \texttt{0xf}\big) = 0, \\
\text{Stage 1b (bits 4..7):}\quad & \big((S \oplus X) \wedge D \wedge \texttt{0xff}\big) = 0 \implies \text{fixes } R_{\text{lo}} \text{ and } (e_{13})_{18} \oplus (e_{13})_{13}, \\
\text{Stage 1c (bits 8..11):}\quad & \big((S \oplus X) \wedge D \wedge \texttt{0xfff}\big) = 0, \\
\text{Stage 1d (bits 12..14 + exact bit 18):}\quad & \big((S \oplus X) \wedge D \wedge \texttt{0x7fff}\big) = 0, \\
\text{Stage 1e (bits 15..17, checked to bit 18):}\quad & \big((S \oplus X) \wedge D \wedge \texttt{0x7ffff}\big) = 0 \implies \text{fixes bits 25..31 via [E21]}, \\
\text{Stage 2a (bits 19..21):}\quad & \big((S \oplus X) \wedge D \wedge \texttt{0x3fffff}\big) = 0, \\
\text{Stage 2b (bits 22..24):}\quad & \big((S \oplus X) \wedge D\big) = 0 \ \wedge \ \big((a_6 \oplus S) \wedge \neg D\big) = 0.
\end{aligned}
\tag{E25}
$$

$$
\text{[Equation (E26) retired in Round 2: former heuristic pre-\texttt{s19} } \Sigma_0(a_9) \text{ window check omitted for 100\% completeness.]} \tag{E26}
$$

In Stage 1d, once bit 13 of e_{13} is chosen, bit 18 of e_{13} is immediately determined by $(R_{\text{lo}} \gg 7) \oplus (e_{13})_0 \oplus (e_{13})_{13}$ (`[E20]`), allowing Stage 1e to test a **19-bit Carry-Sudoku mask (`0x7ffff`)** on 18 chosen bits *before* $\Sigma_1(e_{13,\text{base}})$ is evaluated. Once Stage 1e fixes bits $25..31$ via `[E21]` , Stages 2a and 2b sweep the remaining 6 free bits $19..24$ (`s19` and `s22`) using a 22-bit mask (`0x3fffff`) followed by the full 32-bit Carry-Sudoku check and exact equivalence `[E23]` . *(Note: we do not pre-filter bits* $13..15$ *or* $3..4$ *of* $a_6 - a_5$ *[`[E26]`] before `s19` and `s22` are chosen, because bits* $19..24$ *of* e_{13} *enter* $\text{Maj}(a_9, a_8, a_7)$ *and* $a_9 = e_{13} - c_{e_{13}}$ *with carry propagation; omitting any pre-`s19` window check guarantees 100% completeness across each* 2^{24}*-candidate* a_{12} *block by Theorem 1 (`[E23]` – `[E25]`), confirmed with zero missed contexts against exhaustive brute-force across* 300×2^{24} *candidate* e_{13} *blocks and demonstrated on seed block `a_12 = 0x1ee9c019` [where 2 of 8 exact* $D_{\text{shape}} = 0$ *contexts were rejected by `[E26]`] in `solve20_padded_fast.c` and `verify_all_witnesses.py` §8.)*

```
Algorithm 1: Causal 7-Stage Carry-Sudoku Right-Context Generator
Input: Target terminal words a_{13}..a_{20} = H* - IV, seed range a_{12} in [0, 2^32)
Output: Reservoir R of exact D_shape = (a_5 ^ a_6) == 0 FIPS 180-4 padded contexts
1.  for each seed a_{12} in parallel do
2.    Set e_{15} = 0xffffffff; compute a_{11} via [E15] and set a_{10} = a_{11}            // [E22] Ch-annihilation
3.    Compute constants e_{16}, e_{14}, e_{12} = c_{e12} - e_{14}, and a_8 = const         // [E16]-[E17]
4.    Sweep bits 0..3 and 4..7 of e_{13} filtering ((S ^ X) & D & mask) == 0 (masks 0xf, 0xff)
5.    Compute low-byte padding target R_lo = (c_{W13} - Ch(e_{13}, e_{12}, e_{11}) - 0x80) mod 256  // [E19]
6.    Sweep bits 8..11, 12..14 (fixing bit 18 via [E20]), and 15..17 filtering masks 0xfff, 0x7fff, 0x7ffff
7.    Fix bits 25..31 of e_{13} in O(1) via Delta = (R_lo ^ Sigma_1(e_{13,base})) mod 256   // [E21]
8.    for s_{19} in 0..7 filtering ((S ^ X) & D & 0x3fffff) == 0 do                        // Stage 2a
9.      for s_{22} in 0..7 do                                                              // Stage 2b
10.       if ((S ^ X) & D) == 0 and ((a_6 ^ S) & ~D) == 0 then                             // Theorem 1 [E23]
11.         Append context (a_5..a_{12}, T_{16}, T_{17}, T_{18}, T_{19}) to Reservoir R
```

Across $10{,}000$ random a_{12} blocks, Algorithm 1 evaluates an average of only $43{,}580$ inner-loop nodes per 2^{24} block ($16.0 + 80.4 + 412.9 + 1{,}043.8 + 3{,}507.8 + 8{,}845.4 + 29{,}673.5$), generating 266.8 **exact** $D_{\text{shape}} = 0$ **padded contexts per second** on 16 CPU threads ($3.75\ \text{ms/context}$, `solve20_padded_fast.c`) and $29{,}310$–$114{,}507$ **contexts per second** in supplementary 1× NVIDIA L4 GPU telemetry (`solve20_padded_cuda.cu` , 8.73–$34.1\ \mu\text{s/context}$).

---

6. Slot-Sudoku & Zero-Table Closed-Form $\mathbb{F}_2$ Inverse for Left-to-Right Q_0^{-1} Chains

With an exact $D = a_5 \oplus a_6 = 0$ padded Right context in hand, we solve the three chained Q_0^{-1} equations `[E14]` from Left to Right.

6.1 Closed-Form Zero-Table $\mathbb{F}_2$ Inverse (`reconstruct_u` , `[E27]` – `[E29]`)

Storing full 32-bit preimages u of $y = Q_0(u) = \sigma_0(u) - u$ requires 4 bytes per slot. Splitting $u = U + u_{\text{lo}}$ where $U := (u_{\text{hi}} \ll 16)$ and $u_{\text{lo}}, u_{\text{hi}} \in [0, 65535]$, note that $U \equiv 0 \pmod{2^{16}}$, so carry propagation from bits $0..15$ into bit 16 of $u + y = U + (u_{\text{lo}} + y)$ is purely additive (XOR) at bit 16: $(u + y) \equiv U \oplus (u_{\text{lo}} + y) \pmod{2^{17}}$. Since $u + y \equiv \sigma_0(u) = \sigma_0(U) \oplus \sigma_0(u_{\text{lo}}) \pmod{2^{17}}$, rearranging terms yields the exact 17-bit identity:

$$
\sigma_0(U) \oplus U \equiv (u_{\text{lo}} + y) \oplus \sigma_0(u_{\text{lo}}) \pmod{2^{17}}. \tag{E27}
$$

Rather than inverting `[E27]` solely with a 128 KiB lookup table (`REC_HI16[rec_idx(ulo, y)]` in `solve20_padded_fast.c`), we prove that `[E27]` has an **exact register closed form** with zero lookup tables:

Theorem 2 (Closed-Form $\mathbb{F}_2$ Inverse of Upper 16 Bits). *Let* $v = (u_{\text{lo}} + y) \oplus \sigma_0(u_{\text{lo}}) \pmod{2^{32}}$*, and let* $U = u_{\text{hi}} \ll 16 \in \mathbb{F}_2^{32}$ *have bits in positions* $16..31$ *(with* $U_{0..15} = 0$*). Define the linear operator* $L(T) := T \oplus (T \ll 11) \oplus (T \ll 15) \pmod{2^{32}}$ *on upper-half words* T*. Then* L *is a triangular bijection on bits* $16..31$ *with* $U = L(T)$*, where* T *is given in closed form by:*

$$\begin{aligned} b_0 &= \big((v \ll 15) \oplus (v \ll 11) \oplus v\big) \wedge \texttt{0x00010000}, \\ b_{12} &= \big((v \ll 14) \oplus (v \ll 3)\big) \wedge \texttt{0x00060000}, \\ T &= b_0 \mid b_{12} \mid \big((v \ll 18) \wedge \texttt{0xfff80000}\big), \end{aligned} \tag{E28}$$

and the full 32-bit preimage is recovered in registers (`reconstruct_u_dev` *in* `solve20_padded_cuda.cu` *and* `reconstruct_u_e28` *in* `test_all_12_theories.c`*, using 5 logical stages / ~19 bitwise/ALU operations and zero lookup tables):*

$$u = u_{\text{lo}} \mid \big(T \oplus (T \ll 11) \oplus (T \ll 15)\big). \tag{E29}$$

Proof. By `[E27]` and $U_{0..15} = 0$, $v_b = (\sigma_0(U))_b$ for $b \in \{0, \dots, 15\}$ and $v_{16} = (\sigma_0(U))_{16} \oplus U_{16}$. Since $U_{0..15} = 0$, on bits $0..16$ the three terms of $\sigma_0(U) = \mathrm{ROR}(U, 7) \oplus \mathrm{ROR}(U, 18) \oplus (U \gg 3)$ are:

- $\mathrm{ROR}(U, 7)_b = U_{b+7}$ for $b \in \{9, \dots, 16\}$ and 0 for $b \in \{0, \dots, 8\}$;
- $\mathrm{ROR}(U, 18)_b = U_{b+18}$ for $b \in \{0, \dots, 13\}$ and $U_{b-14} = 0$ for $b \in \{14, 15, 16\}$;
- $(U \gg 3)_b = U_{b+3}$ for $b \in \{13, \dots, 16\}$ and 0 for $b \in \{0, \dots, 12\}$.

Defining $T = L^{-1}(U)$ so that $U_j = T_j$ for $j \in \{16, \dots, 26\}$, $U_j = T_j \oplus T_{j-11}$ for $j \in \{27, \dots, 30\}$, and $U_{31} = T_{31} \oplus T_{20} \oplus T_{16}$ yields:

- For $b \in \{1, \dots, 13\}$, $v_b = (\sigma_0(U))_b = T_{b+18}$, giving $T_{19..31} = (v \ll 18) \wedge \texttt{0xfff80000}$ (in particular, $v_1 = U_{19} = T_{19}$, $v_3 = U_{21}$, $v_4 = U_{22}$, and $v_5 = U_{23}$).
- For $b \in \{14, 15\}$, $v_{14} = U_{21} \oplus U_{17} = v_3 \oplus T_{17}$ and $v_{15} = U_{22} \oplus U_{18} = v_4 \oplus T_{18}$, isolating $T_{17..18} = \big((v \ll 14) \oplus (v \ll 3)\big) \wedge \texttt{0x00060000}$.
- For $b = 16$, $(\sigma_0(U))_{16} = \mathrm{ROR}(U, 7)_{16} \oplus \mathrm{ROR}(U, 18)_{16} \oplus (U \gg 3)_{16} = U_{23} \oplus U_2 \oplus U_{19} = v_5 \oplus 0 \oplus v_1$. Since $v_{16} = (\sigma_0(U))_{16} \oplus U_{16} = v_5 \oplus v_1 \oplus T_{16}$, we obtain $T_{16} = v_1 \oplus v_5 \oplus v_{16}$, which at bit position 16 is $b_0 = \big((v \ll 15) \oplus (v \ll 11) \oplus v\big) \wedge \texttt{0x00010000}$.

Proved algebraically above for all $u \in \mathbb{Z}_{2^{32}}$ and verified empirically on 10^6 pseudorandom words plus 3×2^{16} systematic boundary/carry cases (including $u = \texttt{0x00400000}$) in `test_all_12_theories.c` (`test_theory_01`) with 0 mismatches. □

6.2 Slot-Sudoku: 6.00 GiB 2-Tier Nested 4-Slot EDF Table & 8.00 GiB / 16.00 GiB / 32.00 GiB Direct Tables (`[E30]` – `[E33]`)

Over all 2^{32} inputs $u \in \mathbb{Z}_{2^{32}}$, the exact, race-free preimage multiplicity histogram of $Q_0(u) = \sigma_0(u) - u \pmod{2^{32}}$ is:

- $N_0 = 1{,}573{,}363{,}668$ (36.6327% of y have 0 roots, vs. $\mathrm{Poisson}(1)\, e^{-1} = 36.7879\%$),
- $N_1 = 1{,}586{,}605{,}819$ (36.9410%), $N_2 = 793{,}515{,}119$ (18.4755%), $N_3 = 262{,}185{,}142$ (6.1045%),
- $N_4 = 64{,}407{,}366$ (1.4996%), $N_5 = 12{,}552{,}261$ (0.2923%), $N_6 = 2{,}022{,}487$ (0.0471%),
- $N_7 = 277{,}772$ (0.0065%), $N_8 = 33{,}662$ (0.0008%), and $N_{\geq 9} = 4{,}000$ (0.0001%; specifically $N_9 = 3{,}624$, $N_{10} = 336$, $N_{11} = 36$, $N_{12} = 3$, $N_{13} = 0$, $N_{14} = 1$, with tail root weight $\sum_{k=9}^{14} k N_k = 36{,}422$),

satisfying the exact conservation identities $\sum_{k=0}^{14} N_k = 2^{32} = 4{,}294{,}967{,}296$ and $\sum_{k=0}^{14} k N_k = 2^{32} = 4{,}294{,}967{,}296$. To avoid conflating distinct sources of root loss, we separate three effects: (i) **unconstrained C-slot capacity** $\sum_{k=0}^{14} \min(k, C) N_k / 2^{32}$, which equals $3{,}856{,}601{,}437/2^{32} = 89.7935\%$ for $C = 2$ (72.399% 3-level survival), $4{,}198{,}084{,}127/2^{32} = 97.7443\%$ for $C = 3$ (93.384% 3-level), and $4{,}277{,}381{,}675/2^{32} = 99.5906\%$ for $C = 4$ (98.777% 3-level); (ii) `u_lo = 0` **empty-slot sentinel exclusion** ($1/65{,}536 \approx 0.0015\%$ of inputs have $u \bmod 2^{16} = 0$, shifting the stored root count by at most $65{,}536$, e.g. to 89.7924% for $C = 2$, alongside historical pre-CAS finite-pass table builds that recorded $3{,}856{,}410{,}516/2^{32} = 89.7891\%$ for $C = 2$ and $4{,}276{,}521{,}180/2^{32} = 99.5705\%$ for $C = 4$); and (iii) **quad-shared 12-slot sliding/cuckoo eviction** in the compact $6.00\ \mathrm{GiB}$ format. By Theorem 2, storing only the lower 16 bits $u_{\text{lo}} = u \bmod 2^{16}$ halves the memory footprint of every table tier:

1. **6.00 GiB 2-Tier Nested 4-Slot EDF + Cuckoo-Sudoku Table (`solve20_padded_fast.c`):** Designed for 16 GiB laptops (3×2^{30} entries of `uint16_t` = `6.00 GiB`), where every 16 consecutive y values (a **quad** $q = y \gg 4$) share **12 contiguous 16-bit slots**:

- **Earliest-Deadline-First (EDF) 3-Slot Sliding Window (`[E30]`):** At Levels 1 and 2, each y probes a 3-slot window within its 12-slot quad at offset:

$$\mathrm{slide3_off}(y) = (y \gg 1) + (y \gg 2) - \big(\mathrm{ROR}(\texttt{0x6AAA6AAA}, y \wedge 31) \gg 31\big). \tag{E30}$$

- **Cuckoo-Sudoku Eviction & Nested 4th Slot (`[E31]`):** During construction, a cuckoo-relocation pass (`sudoku_upgrade_worker_6gb` and `tier2_nest4_quad_worker`) packs displaced preimages across each 12-slot quad, placing any 4th root at $\mathrm{b}[(y \wedge 15) \geq 13 \ ? \ -1 : 3]$ (probed only

when all 3 primary slots are valid roots of y). At Level 3, all 12 slots of the quad (`3 × uint64_t`) are tested simultaneously via 64-bit SWAR (`[E35]`):

$$\text{6.00 GiB Coverage} = \frac{3{,}563{,}795{,}213}{2^{32}} = 82.976\%, \quad \text{3-Level Survival} = (0.82976)^3 = 57.13\%. \tag{E31}$$

2. **8.00 GiB / 16.00 GiB Direct-Indexed 2-Way GPU VRAM Tables (`solve20_padded_cuda.cu`):** Built natively in GPU VRAM in `3.11 seconds` , storing up to two roots per y (unconstrained 2-way capacity $3{,}856{,}601{,}437/2^{32} = 89.7935\%$ / 3-level survival 72.399%, or 89.7924% with `u_lo = 0` sentinel exclusion):

$$\text{2-Way Direct Capacity} = \frac{3{,}856{,}601{,}437}{2^{32}} = 89.7935\%, \quad \text{3-Level Survival} = (0.897935)^3 = 72.399\%. \tag{E32}$$

3. **32.00 GiB Direct-Indexed 4-Way Exact CPU Table (`H_exact32[y]` , `--table-gb 32`):** Stores up to four 16-bit roots per y in one `uint64_t` (`32.00 GiB` table + `4.00 GiB` count array + `16.00 GiB` optional F_1 table = `52.00 GiB` persistent RAM, `68.00 GiB` peak during parallel radix sort), achieving 99.5906% unconstrained 4-way capacity (98.777% 3-level survival, or $4{,}276{,}521{,}180/2^{32} = 99.5705\%$ / 98.717% 3-level in the pre-CAS `u_lo = 0` -excluded build):

$$\text{4-Way Direct Capacity} = \frac{4{,}277{,}381{,}675}{2^{32}} = 99.5906\%, \quad \text{3-Level Survival} = (0.995906)^3 = 98.777\%. \tag{E33}$$

6.3 The 3-Level Tree Search, 64-Bit SWAR Level-3 Filter, and Terminal 20-Step Bijection (`[E34]` – `[E37]`)

For each a_1 and each Right context in a batch of size B:

- **Level 1 ($a_1 \to W_1 \to a_2$) & Level 2 ($a_2 \to W_2 \to a_3$) with Early Hamming Pruning (`[E34]`):** From a_1, look up $W_1 \in Q_0^{-1}(T_{16} + F_1(a_1))$, set $a_2 = W_1 + G_1(a_1)$, look up $W_2 \in Q_0^{-1}(T_{17} + G_2(a_1, a_2) - W_1)$, and set $a_3 = W_2 + G_2$. Before computing e_3, G_3 or issuing Level-3 DRAM loads, we prune any a_3 whose majority mask $M_3 = \neg(a_3 \oplus a_5)$ has Hamming weight exceeding $w_{\text{thresh}} \in \{13, 14\}$:

$$\text{popcount}\big(\neg(a_3 \oplus a_5)\big) \le w_{\text{thresh}}, \tag{E34}$$

  eliminating 81.15% ($w_{\text{thresh}} = 13$, **exact** $\text{Binomial}(32, 1/2)$ **upper tail** $\Pr[\text{Binomial}(32, 1/2) > 13]$) **to** 70.17% ($w_{\text{thresh}} = 14$) of Level-3 lookups.

- **Level 3: 64-Bit SWAR Pre-Reconstruction Filter & 19-Step Bridge Candidate Stream (`[E35]`):** Because the majority condition $\big((a_4 \oplus a_5) \wedge M_3\big) = 0$ (`[E13]`) is lower-triangular on bits $0..14$, any candidate u_{lo} must satisfy $\big(((u_{\text{lo}} + G_3) \oplus a_5) \wedge M_3 \wedge \texttt{0x7fff}\big) = 0$ **before** u_{hi} is reconstructed. In the 6.00 GiB table, we test 4 slots at a time in a single 64-bit word `w` using SWAR carry-bit detection (`K7 = 0x7fff7fff7fff7fff` , `K8 = 0x8000800080008000`):

$$d = \Big(\big(((w \wedge K_7) + g_{64}) \oplus A_{64}\big) \wedge m_{64}\Big) + K_7, \qquad \text{4-slot hit mask} = \neg d \wedge K_8, \tag{E35}$$

  rejecting $> 99\%$ of 12-slot quads in 3 register operations. Every candidate passing the full 32-bit check $\big((a_4 \oplus a_5) \wedge M_3\big) = 0$ with $Q_0(W_3) = t_{18}$ is a **schedule-consistent 19-step padded bridge candidate** (`sol19` , incremented via `local_sol19++`) satisfying W_{16}, W_{17}, W_{18} for the current right-hand context.

- **Terminal 20-Step Filter & Exact $\mathbb{F}_2^{32}$ Bijection (`[E36]` – `[E37]`):** In the 20-step solver (where $a_{13..20}$ are pinned by H_{20}^*), for each `sol19` bridge candidate we recover W_4 via `[E7]` and test the step-19 schedule equation:

$$\sigma_0(W_4) + W_3 \equiv T_{19} \pmod{2^{32}}, \qquad T_{19} := W_{19} - \sigma_1(W_{17}) - W_{12}. \tag{E36}$$

  Viewing $\sigma_0(x) = \text{ROR}(x, 7) \oplus \text{ROR}(x, 18) \oplus (x \gg 3)$ as a 32×32 matrix over $\mathbb{F}_2$, we have $\det_{\mathbb{F}_2}(\sigma_0) = 1$:

$$\det_{\mathbb{F}_2}(\sigma_0) = 1 \implies W_4 = \sigma_0^{-1}\big((T_{19} - W_3) \bmod 2^{32}\big) \text{ is uniquely determined by } W_3 \text{ and } T_{19}. \tag{E37}$$

6.4 Exact-T_{16} Twin Coalescing, L2 Cache Tiling, and Lower-Triangular Pruning (`[E42]` – `[E45]`)

To maximize memory-system throughput across Levels 1–3 on GPU (`solve20_padded_cuda_prod.cu` , `solve20_padded_cuda.cu`), we combine four techniques (formalized in **Algorithm 2**):

1. **L2-Localized Level-1 Bridge Extraction (`extract_l1_bridges_kernel` , `[E42]`):** The NVIDIA L4 GPU features a 48 MiB L2 cache. Because $F_1(a_1) = G_1(a_1) - W_0(a_1)$ includes the 32-bit rotation $\Sigma_1(e_1(a_1))$, F_1 is **not** lower-triangular in the 2-adic bit order of a_1 (for example, $a_1 = 0 \implies F_1 = \texttt{0d3fffb7}$ whereas $a_1 = 4 \implies F_1 = \texttt{3ce015b4}$, which differ in bit 0). Instead of assuming lower-triangularity of F_1, `extract_l1_bridges_kernel` partitions Q_0^{-1} global-memory lookups into address tiles $[\text{Tile}_{\text{lo}}, \text{Tile}_{\text{hi}})$ of width $\Delta_{\text{tile}} = \text{Tile}_{\text{hi}} - \text{Tile}_{\text{lo}}$ (at 4 bytes per `uint32_t` entry in the 16.00 GiB 2-Way table, an L2-resident 16–32 MiB tile spans $\Delta_{\text{tile}} = 2^{22}$–$2^{23}$ indices across

512–1024 tile passes, whereas $\Delta_{\text{tile}} = 2^{29}$ indices spans a 2.00 GiB slice across 8 passes) and tests the target index $t_{16}(a_1) = (T_{16} + F_1(a_1)) \bmod 2^{32}$ in registers before issuing a global memory read:

$$(T_{16} + F_1(a_1) - \text{Tile}_{\text{lo}}) \bmod 2^{32} < \Delta_{\text{tile}} \implies \text{under a uniform index model, a fraction } \frac{\Delta_{\text{tile}}}{2^{32}} \text{ issues a global read per tile pass} \tag{E42}$$

converting uncoalesced 16.00 GiB DRAM accesses into localized tile passes across the sweep (without reducing total work summed over all passes).

2. **Exact 32-Bit T_{16} Twin Coalescing (`twin2`) vs. 4-Way Reservoir Sampling (`single4`, `[E43]`):** Across all 2^{32} seeds a_{12}, a 20-step target H_{20}^* has an estimated universe of $U \approx 1.68 \times 10^7 \approx 2^{24.00}$ exact $D_{\text{shape}} = 0$ right contexts (or $U_{\text{filt}} \approx 1.014 \times 10^7 \approx 2^{23.27}$ if the optional Stage-2 heuristic is enabled). Sorting a reservoir of contexts by T_{16} yields $N_{\text{twins}} \approx U^2/2^{33} \approx 32{,}890$ **exact 32-bit T_{16} twin pairs** across U (or $\approx 11{,}970$ pairs in U_{filt}) where $T_{16}^{(2m)} = T_{16}^{(2m+1)}$ (`sweep_exact16_cuda_kernel` in `solve20_padded_cuda_prod.cu` and `solve20_padded_cuda.cu`). Both contexts in each genuine twin pair share the exact same Level-1 lookup $Q_0^{-1}(T_{16} + F_1(a_1))$ (`0.50` Level-1 lookups/context), solving 12 of our 16 padded 20-step targets directly. For the remaining 4 targets (16/16 total), the non-twin 4-way reservoir kernels (`sweep_20_warp_l3_prod_kernel` in `solve20_padded_cuda_prod.cu` and `sweep_exact16_cuda_kernel` in `solve20_padded_cuda.cu`) sample from the reservoir U using 4-way register-prefetched Level-1 batches (achieving $53{,}847.5$–$73{,}560.5$ sol19/s on the 6.00 GiB table and $132{,}888.7$ sol19/s on genuine Exact-T_{16} twin pairs in the 16.00 GiB table on 1× L4 GPU):

$$T_{16}^{(2m)} = T_{16}^{(2m+1)} \quad \text{across } M = 64 \text{ genuine twin pairs } (B = 128) \implies \mathbf{0.50} \text{ L1 lookups per context.} \tag{E43}$$

(Note: whereas 32-bit T_{16} collisions occur naturally in $U \approx 2^{24.0}$, joint 64-bit (T_{16}, T_{17}) collisions require a birthday pool of $2^{32} \gg U$ contexts; all witness runs and primary complexity figures in this paper use only genuine, unmutated contexts.)

3. **Pre-`ulo2` 16-Bit Lower-Triangular Hamming Pruning (`[E44]`):** Before reconstructing the upper 16 bits of W_2 via `[E29]`, we exploit lower-triangularity of modular addition on bits $0..15$:

$$w_{\text{lo},2} := \text{popcount}\Big(\neg\big((u_{\text{lo},2} + G_2) \oplus a_5\big) \wedge \texttt{0xffff}\Big) \le w_{\text{lo,thresh}} \in \{7, 11\}. \tag{E44}$$

For a uniform 16-bit mask, $\text{Binomial}(16, 1/2)$ rejects $39{,}203/65{,}536 \approx 59.82\%$ of Level-2 candidates at $w_{\text{lo,thresh}} = 7$ and $2{,}517/65{,}536 \approx 3.84\%$ at $w_{\text{lo,thresh}} = 11$ prior to upper-half reconstruction.

4. **The 4-Round Crossed Carry-Sudoku Spine ($a_2 = a_3 = a_4 = a_5$, `[E45]`):** Crossing the shaping constraint across 4 consecutive rounds by pinning $a_2 = a_3 = a_4 = a_5$ (`cross_round_4step_sudoku_direct_kernel` in `solve20_padded_cuda.cu`) collapses all four majority functions to a_5 and makes W_1, W_2, W_3 direct functions of a_1 with zero Q_0^{-1} table lookups:

$$a_2 = a_3 = a_4 = a_5 \implies \begin{cases} \text{Maj}(a_3, a_2, a_1) = \text{Maj}(a_4, a_3, a_2) = \text{Maj}(a_5, a_4, a_3) = \text{Maj}(a_6, a_5, a_4) = a_5, \\ e_4 = a_0 - \Sigma_0(a_5), \quad W_3 = c_{W_3} - \Sigma_1(e_3) - \text{Ch}(e_3, e_2, e_1), \\ \text{5-step } (a_1 = a_5) : e_3 = a_{-1} - \Sigma_0(a_5) \implies W_3(a_0) = C_3 - \text{Ch}(e_3, e_2, e_1). \end{cases} \tag{E45}$$

Because $a_2 = a_3 = a_4 = a_5$ spends 96 Left bits, the 3 schedule equations W_{16}, W_{17}, W_{18} become a 96-bit Right filter, illustrating the degree-of-freedom conservation trade-off (§9.2).

```
Algorithm 2: Left-to-Right Slot-Sudoku Tree & Terminal 20-Step Preimage Solver
Input: Context batch C of size B=128 (T_{16} twin-coalesced or 4-way batched), 2-way table H_ulo
Output: Stream of 19-step bridge candidates (sol19) and matching 20-step padded preimage M*
1.  for each root a_1 in [0, 2^32) and context group in C do
2.    Compute F_1(a_1) and G_1(a_1); prefetch Level-1 entries from H_ulo[T_{16} + F_1(a_1)]        // [E14], [E43]
3.    for each valid Level-1 root W_1 do
4.      Set a_2 = W_1 + G_1(a_1); compute G_2(a_1, a_2); load u_{lo,2} from H_ulo[T_{17} + G_2 - W_1]
5.      if popcount((~((u_{lo,2} + G_2) ^ a_5)) & 0xffff) <= w_{lo,thresh} then                    // [E44]
6.        Reconstruct W_2 via [E28]-[E29]; set a_3 = W_2 + G_2 and M_3 = ~(a_3 ^ a_5)
7.        if popcount(M_3) <= 14 then                                                            // [E34]
8.          Load u_{lo,3} from H_ulo[T_{18} + G_3 - W_2]; filter (((u_{lo,3}+G_3)^a_5) & M_3 & 0x7fff)==0 // [E35]
9.          Reconstruct W_3 via [E28]-[E29]; set a_4 = W_3 + G_3
10.         if ((a_4 ^ a_5) & M_3) == 0 then                                                       // [E13]
11.           Emit 19-step bridge candidate (sol19); compute W_4 via [E7]
12.           if sigma_0(W_4) + W_3 == T_{19} (mod 2^32) then return 20-step padded preimage M*    // [E36]
```

6.5 Precomputation vs. Online Time, Memory, and SHA-256-Equivalent Complexity

Following standard cryptographic convention [2, 3, 4, 8], Table 6.1 separates **target-independent precomputation** (Q_0^{-1} table construction) from **target-dependent online complexity**, reported both in direct wall-clock time (`node-h` denoting wall-clock hours on one 16-thread CPU node) and against

our measured same-host **10-core CPU reference baseline** (1.5125×10^8 SHA20-eq/s, measured on arbitrary 20-step message blocks, so 1 node-h $= 2^{38.99}$ CPU-ref). To avoid conflating CPU work factors with cross-hardware elapsed-time conversions, we report:

1. **Host-Verifiable CPU Solver (`solve20_padded_fast.c`, 10-core Apple M4, 16 threads):** Generates $6{,}242.8$ sol19/s (160.2 μs/sol19, measured on a_1 slice sweeps) on genuine unmutated contexts. Against the same-host CPU reference baseline (1.5125×10^8 SHA20-eq/s), each `sol19` costs $24{,}227.9 \approx 2^{14.56}$ CPU-ref evaluations, and an expected 20-step padded preimage (2^{32} sol19 $= 191.1$ node-h on 1 16-thread CPU node, or 23.9 node-h across 8 CPU nodes) costs $2^{46.56}$ CPU-ref evaluations.
2. **Supplementary GPU Slice Telemetry (`solve20_padded_cuda.cu` / `solve20_padded_cuda_prod.cu`, 1× NVIDIA L4 GPU):** On genuine unmutated contexts, measured a_1-slice sweep telemetry (`BENCH_4_IDEAS_RESULTS.txt` and `solve20_padded_cuda.cu`) records $53{,}847.5$ sol19/s (Cycle 0, 6.00 GiB 2-Tier EDF, $B = 2$, 22.16 GPU-h) to $73{,}560.5$ sol19/s (Cycle 1, 6.00 GiB 2-Tier EDF, $B = 8$, 16.22 GPU-h per 2^{32} sol19 on 1× L4), and $132{,}888.7$ sol19/s on genuine Exact-T_{16} twin pairs in the 16.00 GiB Direct 2-Way table (`sweep_exact16_cuda_kernel` in `solve20_padded_cuda_prod.cu` and Cycle 1 of `solve20_padded_cuda.cu`, 8.98 GPU-h on 1× L4, or 1.12 h on 8× L4). *(The experimental `--bench-c2` through `--bench-c10` harnesses [Cycles 2, 4, and 7–10 in `solve20_padded_cuda.cu`, including the* $180{,}831.9$ sol19/s / 6.60 GPU-h $B = 16$ *shared-T_{16} run] are architectural microbenchmarks and are excluded from single-target complexity claims; likewise, the* 9.62×10^9 evals/s *GPU rate in Table 9.4 Row 13 is the specialized 4-round crossed-spine kernel rather than an arbitrary-message 20-step SHA-256 baseline.)*
3. **Chosen-Target 19-Step Padded Hash (`src/pre19.c`, 1 CPU core):** `src/pre19.c` is a single-threaded C solver using `clock()` with default `m = 29` (2^{29} entries), where the two-pass CSR builder performs $2 \times 2^{29} = 2^{30}$ Q_0 evaluations in 16.8–19.1 s and allocates 4×2^{29} B $= 2.0$ GiB for `table_values` plus 64 MiB for `table_offsets` (2.0625 GiB persistent RAM). Across our 14 public-label 19-step targets, each solve evaluates $\approx 2^{26.7}$–$2^{26.9}$ right-context trials per usable context plus $\approx 1.8 \times 10^6$–1.2×10^8 ($2^{20.8}$–$2^{26.8}$, median $\approx 2^{25.5}$ at $m = 29$) left chain starts, with total single-core online search time ranging from 1.2 s **to** 86.4 s (median 19.45 s, mean 30.2 s, corresponding to $2^{32.51}$–$2^{32.87}$ 19-step evaluations).

Table 6.1: Precomputation and online complexities of the proposed 19- and 20-step preimage attacks on genuine contexts.

Attack & Target Setting	Precomputation Time	Memory Footprint	Online Context Gen	Online Search (`CPU-ref` / `sol19`)	Wall-Clock Online Time
19-Step Padded Hash (Chosen H^*_{19}, `src/pre19.c`)	2×2^{29} Q_0 evals (16.8–19.1 s, 1 core)	2.0625 GiB (`m=29` CSR)	$\approx 2^{26.7\text{–}26.9}$ trials/ctx ($D \le 3$)	$2^{20.8\text{–}26.8}$ **chains (** $2^{32.51\text{–}32.87}$ **evals)**	**`1.2–86.4 s` (median `19.45 s`, 1 CPU core)**
19-Step Bridge Stream (`sol19`) (`solve20_padded_fast.c`)	2^{32} Q_0 evals (3.11 s, 16 threads)	6.00 GiB (CPU) / 6–16 GiB (GPU)	266.8 ctx/s (CPU)	$24{,}228$ ($2^{14.56}$) **`CPU-ref` / `sol19`**	**`160.2 µs/sol19` (CPU)**; *supp. GPU:* `7.5–18.6 µs`
20-Step Padded Hash (Chosen H^*_{20}, 55B / 119B)	2^{32} Q_0 evals (3.11 s, 16 threads)	6.00 GiB (CPU) / 6–16 GiB (GPU)	266.8 ctx/s (CPU)	$2^{46.56}$ CPU-ref (2^{32} sol19 on CPU)	**`191.1 node-h` (1× CPU)**; *supp. GPU:* **`8.98–22.16 h` (1× L4)**
20-Step Unpadded Compression (§7, `solve20_zero_a3.c`)	3×2^{32} evals (45 s, 16 threads)	48.00 GiB RAM	$O(1)$ per (a_{11}, a_{12})	$2^{38.13}$ CPU-ref expected (2^{32} sol19)	33.14 min **exp.** / **`21.10 min` obs.** (16 CPU cores)

7. Companion Result: 20-Step Unpadded Compression Preimages via the Zero-a_3 Bridge

When FIPS 180-4 padding is not required (the 20-step compression-function model studied in [6, 7, 9]), we can eliminate the $(3/4)^{32} \approx 2^{-13.28}$ (13.28-bit) majority survival loss of `[E13]` altogether by placing the state on the **Zero-a_3 Manifold** (`solve20_zero_a3.c`), where the middle state words are pinned to:

$$a_3 = 0, \quad a_5 = 0, \quad a_6 = 0, \quad a_8 = 0, \quad a_9 = 0, \quad a_{10} = \texttt{0xffffffff} \implies e_9 = 0, \quad e_{10} = \texttt{0xffffffff}. \tag{E38}$$

Because $\mathrm{Maj}(0, x, 0) = 0$, both majority functions containing a_4 vanish identically for 100% **of** $a_4 \in [0, 2^{32})$:

$$\mathrm{Maj}(a_5, a_4, a_3) = \mathrm{Maj}(0, a_4, 0) = 0, \qquad \mathrm{Maj}(a_6, a_5, a_4) = \mathrm{Maj}(0, 0, a_4) = 0. \tag{E39}$$

Fixing $a_3 = 0$ removes one degree of freedom on the Left, which we restore on the Right by leaving a_7 free. Under `[E38]`, $e_6 = a_2$ and $\mathrm{Ch}(e_9, e_8, e_7) = \mathrm{Ch}(0, e_8, e_7) = e_7 = a_7$, so $W_9 = W_{9,0} - a_2 - a_7$ (where $W_{9,0} := W_9|_{a_2 = a_7 = 0}$). Simultaneously, on the Left $a_2 = W_1 + G_1(a_1)$ (with $G_1(a_1) := -W_1|_{a_2=0}$), and on the Right $W_{14} = D_{14}(a_{11}, a_{12}) - a_7$ (where $D_{14}(a_{11}, a_{12}) := W_{14}|_{a_7=0}$). Substituting $-a_2 = -W_1 - G_1(a_1)$ and $-a_7 = W_{14} - D_{14}(a_{11}, a_{12})$ into the step-16 schedule equation $W_{16} = \sigma_1(W_{14}) + W_9 + \sigma_0(W_1) + W_0(a_1)$ groups the W_1 terms into $\sigma_0(W_1) - W_1 = Q_0(W_1)$ and the W_{14} terms into $\sigma_1(W_{14}) + W_{14} = Q_1(W_{14})$. Introducing

a fixed meeting constant $T_{16}^{*} = \texttt{0x12345678}$ decouples step 16 into independent Left (Q_0) and Right (Q_1) table lookups (`phase1_left_worker` and `phase2_right_worker` in `solve20_zero_a3.c`):

$$\begin{aligned} Q_0(W_1) &= T_{16}^{*} - W_0(a_1) + G_1(a_1), & L_{17}(a_1) &:= \sigma_0(W_2) + W_1, \\ Q_1(W_{14}) &= W_{16}(a_{11}, a_{12}) + D_{14}(a_{11}, a_{12}) - W_{9,0} - T_{16}^{*}, & a_7 &= D_{14}(a_{11}, a_{12}) - W_{14}. \end{aligned} \tag{E40}$$

Precomputing 16 GiB tables for Q_0^{-1}, Q_1^{-1}, and the Left bridge $L_{17}(a_1) \mapsto a_1$ satisfying $a_3 = 0$ (48.0 GiB total, 45 s one-time build, 100% independent of H^{*}), `solve20_zero_a3.c` achieves **zero β-filter loss** (63.37% survival per bridge hit) and **2.16 million exact 19-step compression solutions per second** (`0.46 µs/sol19`) on a 16-vCPU CPU node. At 2.16×10^{6} sol19/s, the expected online search time for 2^{32} 19-step solutions is $1{,}988.4$ s ≈ 33.14 minutes ($\approx 2^{38.13}$ scalar CPU `SHA20-eq` at 151.25 M evals/s, specifically $2^{38.1298}$). For the target `deadbeef...` , the solver found a 20-step unpadded preimage after $2^{32.01}$ outer (a_{11}, a_{12}) trials ($2.735 \times 10^{9} \approx 2^{31.35}$ 19-step solutions):

$$\text{Expected 20-Step Unpadded Online Time} = 1{,}988.4 \text{ s } (33.14 \text{ min}); \quad \text{Observed Run} = 1{,}266.26 \text{ s } (21.10 \text{ min, 16 CPU cores}) \tag{E41}$$

---

8. Verified 19-Step and 20-Step Padded SHA-256 Preimage Witnesses

All **46 distinct** 20-step and 19-step witnesses below (verified via 48 checks in `verify_all_witnesses.py` , where `"abc"` appears in both the flagship demonstration §8.2 and the 14-label benchmark suite §8.4: comprising 16 distinct 20-step padded witnesses, 16 distinct 19-step padded witnesses, 13 distinct 20-step unpadded compression witnesses, and 1 Stage-2 completeness regression witness) are independently verified in under 1 second by running `python3 verify_all_witnesses.py` (`verify_all_witnesses.py`). Complete machine-readable witness files are provided in `demos_20step.txt` , `examples_abc_hello_20step.txt` , `witnesses_20step.txt` , and `FOUND_20ROUND_PREIMAGE.txt` (in display blocks below and in §1.1, 110-hex-character 55-byte messages are wrapped across two 55-character lines for readability, while Tables 8.2 and 8.3 print single-line 110-character hex strings).

8.1 Witness 1: First Demonstrated 20-Step Padded SHA-256 Preimage ($H_{20}^{*} = \texttt{0000...0000}$)

Found by `solve20_padded_cuda.cu` on Sweep 62 (`ctx=5` , after $70{,}661{,}112$ exact 19-step bridge candidates; recorded in `FOUND_20ROUND_PREIMAGE.txt`):

- **Target 20-Step Digest (H_{20}^{*}):** `0000000000000000000000000000000000000000000000000000000000000000`
- **55-Byte Message M (440 bits, big-endian hex, split into two 55-hex-char lines):** `ec531306c85ad4a4ad211d2e1a03ece4d0ad4a0e6b4c2b9436b3bab` `dcbd520597196f52c042b8cd95f1a39f71b58b47b0647c78e65416f`
- **FIPS 180-4 Padded 512-Bit Message Block ($W_0 \ldots W_{15}$):**

```
W[ 0.. 3] = ec531306  c85ad4a4  ad211d2e  1a03ece4
W[ 4.. 7] = d0ad4a0e  6b4c2b94  36b3babd  cbd52059
W[ 8..11] = 7196f52c  042b8cd9  5f1a39f7  1b58b47b
W[12..15] = 0647c78e  65416f80  00000000  000001b8
```

(Note the mandatory FIPS 180-4 padding [1]: $W_{13} \bmod 256 = \texttt{0x80}$, $W_{14} = \texttt{0x00000000}$, and $W_{15} = \texttt{0x000001b8} = 440$ bits.)

Table 8.1: Complete 20-step internal state trajectory (a_t, e_t, W_{t-1}) of the 20-step padded witness for $H_{20}^{*} = \texttt{0000...0000}$.

Step t	Register a_t	Register e_t	Message / Schedule Word W_{t-1}	Algebraic / Carry-Sudoku Invariant Verified
1	`e85b9b53`	`851af5a8`	$W_0 =$ `ec531306`	Swept root $a_1 \in [0, 2^{32})$
2	`47d401c6`	`4b125b48`	$W_1 =$ `c85ad4a4`	Level-1 Q_0^{-1} hit: $Q_0(W_1) = T_{16} + F_1(a_1)$
3	`0a280ce0`	`8b761050`	$W_2 =$ `ad211d2e`	Level-2 Q_0^{-1} hit; $\text{popcount}(M_3) = 12 \le 14$
4	`67298bc8`	`c121961d`	$W_3 =$ `1a03ece4`	Level-3 Q_0^{-1} hit; $((a_4 \oplus a_5) \wedge M_3) = 0$ (`[E13]`)
5	**`67978bcd`**	`2724a33e`	$W_4 =$ `d0ad4a0e`	**Exact Shaping (`[E11]`):** $a_5 = a_6 =$ `67978bcd` $(D = 0)$
6	**`67978bcd`**	`eea015fa`	$W_5 =$ `6b4c2b94`	**Exact Shaping (`[E11]`):** $a_6 = a_5 =$ `67978bcd` $(D = 0)$
7	`41b1f1b0`	`8aa086f2`	$W_6 =$ `36b3babd`	Closed-form a_7 absorbing $W_{14} = 0$ (`[E17]`)
8	`edb02749`	`9319765f`	$W_7 =$ `cbd52059`	Invariant a_8 absorbing $W_{15} = 440$ (`[E17]` , `[E22]`)
9	`7fd85be0`	**`ffffffff`**	$W_8 =$ `7196f52c`	**Selector Saturation (`[E8]`):** $e_9 =$ `0xffffffff`

Step t	Register a_t	Register e_t	Message / Schedule Word W_{t-1}	Algebraic / Carry-Sudoku Invariant Verified
10	**a7b351d7**	**ffffffff**	$W_9 =$ 042b8cd9	**Selector Saturation ([E8]):** $e_{10} =$ 0xffffffff, $a_{10} = a_{11}$
11	**a7b351d7**	4f9e3855	$W_{10} =$ 5f1a39f7	**Majority Simplification ([E22]):** $a_{11} = a_{10} =$ a7b351d7
12	e5d45365	81ba7166	$W_{11} =$ 1b58b47b	Right context seed a_{12}
13	bf9f516c	e4acad89	$W_{12} =$ 0647c78e	Target-fixed a_{13}; e_{13} solves [E21] & [E23]
14	b4322266	da868879	$W_{13} =$ 65416f80	**Padding byte ([E21]):** $W_{13} \bmod 256 =$ 0x80
15	8493b564	**ffffffff**	$W_{14} =$ 00000000	Ch-**Annihilation ([E22]):** $e_{15} =$ 0xffffffff, $W_{14} = 0$
16	97457a44	03b1406f	$W_{15} =$ 000001b8	**Padding length ([E1]):** $W_{15} = 440$ bits (0x1b8)
17	5ab00ac6	a41f32e7	$W_{16} =$ d6317d0a	Level-1 schedule word W_{16}; fixed by $H^*_{20} - \mathrm{IV}$
18	c3910c8e	e07c2655	$W_{17} =$ 3815d972	Level-2 schedule word W_{17}; fixed by $H^*_{20} - \mathrm{IV}$
19	4498517b	64fa9774	$W_{18} =$ 89d90bef	Level-3 schedule word W_{18}; 19-step bridge complete
20	95f61999	aef1ad81	$W_{19} =$ cd1c5aee	Terminal hit $\sigma_0(W_4) + W_3 = T_{19} \implies H_{20} =$ 00..00

8.2 Witness 2: Flagship 20-Step & 19-Step Single-Block Padded Preimages for $H^* =$ 0000…0000, SHA-256(“abc”), and SHA-256(“hello”)

Recorded in `demos_20step.txt`, `examples_abc_hello_20step.txt`, and `verify_all_witnesses.py` (55-byte hex strings split into two 55-hex-char lines):

- **Target Digest $H^*_{19} =$ 0000…0000 (All-Zero 256-Bit Digest, 19-Step Padded with Exact $a_5 = a_6$):** `0000000000000000000000000000000000000000000000000000000000000000`
- **19-Step Padded 55-Byte Preimage ($H_{19}(M) =$ 0000…0000):** `fc0c9d8eafb2fa329af316940c42a5eb9773cec39e4f7e396182628` `7ae767638f92d4ce5fc5eb8cabd7606409364dda27a6c1bb5220246`

  ```
  W[ 0.. 7] = fc0c9d8e afb2fa32 9af31694 0c42a5eb 9773cec3 9e4f7e39 61826287 ae767638
  W[ 8..15] = f92d4ce5 fc5eb8ca bd760640 9364dda2 7a6c1bb5 22024680 00000000 000001b8
  ```

  (Invariants verified: $a_5 = a_6 =$ 3dbb85b8 $[D = 0]$, $e_9 = e_{10} =$ 0xffffffff, $W_{13} \bmod 256 =$ 0x80, $W_{14} = 0$, $W_{15} = 440$.)
- **Target Digest SHA-256(“abc”) (FIPS 180-4 §B.1 [1]):** `ba7816bf8f01cfea414140de5dae2223b00361a396177a9cb410ff61f20015ad`
- **20-Step Padded 55-Byte Preimage ($H_{20}(M) =$ SHA-256(“abc”)):** `edf6e70a7001ae71b900f18119aaa1fdf6c1179b4268891262a9539` `be04d41236685e0c102e0ad23165e8b7157669819302ddcb3c68226`

  ```
  W[ 0.. 7] = edf6e70a 7001ae71 b900f181 19aaa1fd f6c1179b 42688912 62a9539b e04d4123
  W[ 8..15] = 6685e0c1 02e0ad23 165e8b71 57669819 302ddcb3 c6822680 00000000 000001b8
  ```

  (Invariants verified: $a_5 = a_6 =$ 4ade1afa, $a_{10} = a_{11} =$ acf3c908, $e_9 = e_{10} = e_{15} =$ 0xffffffff.)
- **19-Step Padded 55-Byte Preimage ($H_{19}(M) =$ SHA-256(“abc”)):** `8ca254f8d04dd3fb04efaad6b1867588ffe833de49a144fedbf1839` `422892dc9f5a9158d8fc063290a14a9a6c55dfa245642a0247ec45a`
- **Target Digest SHA-256(“hello”):** `2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824`
- **20-Step Padded 55-Byte Preimage ($H_{20}(M) =$ SHA-256(“hello”)):** `3a11bd11262fcee01e4a184372643bdbe4ba65651b730b0b8fe49c7` `648866d52c60847db2400ef72c859e2b0366e470fae3cf0891313b4`
- **19-Step Padded 55-Byte Preimage ($H_{19}(M) =$ SHA-256(“hello”)):** `ac055aaa83e1e51a5d74102b7739b7b6e019e31100cd525baf09c64` `2d4b59b8fc4eb8d481df111aee2616325b336b827190103c33584c5`

8.3 Witness 3: Flagship 20-Step & 19-Step Two-Block (119-Byte, Plain-ASCII First Block) Padded Preimages ($H^* =$ SHA-256(“”))

Demonstrating full Merkle–Damgård multi-block chaining with an arbitrary human-readable 64-byte plain-ASCII first block (`"This first block is plain text; the second one was computed....."`) followed by a 55-byte computed second block with FIPS 180-4 padding [1] ($W^{(2)}_{13} \bmod 256 =$ 0x80, $W^{(2)}_{14} = 0$, $W^{(2)}_{15} =$ 0x000003b8 $= 952$ bits, recorded in `demos_20step.txt`):

- **Target Digest ($H^* =$ SHA-256(“”)):** `e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855`

- **Block 1 (64 Bytes Plain ASCII, identical for both 20-step and 19-step demos):** `"This first block is plain text; the second one was computed....."`

```
W^(1)[ 0.. 7] = 54686973 20666972 73742062 6c6f636b 20697320 706c6169 6e207465 78743b20
W^(1)[ 8..15] = 74686520 7365636f 6e64206f 6e652077 61732063 6f6d7075 7465642e 2e2e2e2e
```

- **20-Step Block 2 (55 Computed Bytes + 9-Byte FIPS 180-4 Padding, $L = 952$ bits $=$ `0x3b8`, chaining from $H_{20}^{(1)} =$ `9743298d4cdedcbb11eab6931270fe130dd0420fff6255433a223586f586f980`):**

```
W_20^(2)[ 0.. 7] = df28e21b b6b61202 ba87baa7 2096ee0a 3ac325cf 1f12056e 7cfe2ae1 735b4d10
W_20^(2)[ 8..15] = 6b8f6e0b 22eb9b50 4ed93689 91944b0a 115a934f 17c2c480 00000000 000003b8
```

- **19-Step Block 2 (55 Computed Bytes + 9-Byte FIPS 180-4 Padding, $L = 952$ bits $=$ `0x3b8`):**

```
W_19^(2)[ 0.. 7] = fe409cc9 30128a72 0481f86c 0503d7a0 18519a12 9df44913 9720bb5e ef520895
W_19^(2)[ 8..15] = f8b51568 5bb40481 7be30188 0f0caabe 02fc9e5f 1c95c080 00000000 000003b8
```

8.4 Witness 4: Complete 20-Step and 19-Step Padded Benchmark Suites (14 Targets Each) & Unpadded Suite (13 Targets)

Matching every public-label target in the benchmark suite ($H^* = \text{SHA-256}(\text{label})$), **Table 8.2** lists the complete, untruncated 55-byte (110-hex-character) **20-step FIPS 180-4 padded preimages** (`[padded]` , where appending `8000000000000001b8` forms the 512-bit block $W_0 \dots W_{15}$), and **Table 8.3** lists the corresponding complete 55-byte **19-step FIPS 180-4 padded preimages**.

Table 8.2: Complete 20-step single-block FIPS 180-4 padded benchmark suite (all 14 public-label targets, full 55-byte `message_hex` , $L = 440$ bits).

Target Label (`H* = SHA-256(label)`)	Full 256-Bit Target Digest `H*` (64 hex chars)	Complete 20-Step Padded 55-Byte Preimage `message_hex` (110 hex chars; pad suffix `8000000000000001b8`)	Verified
`"abc"`	`ba7816bf8f01cfea414140de5dae2223b00361a396177a9cb410ff61f20015ad`	`edf6e70a7001ae71b900f18119aaa1fdf6c1179b4268891262a9539be04d41236685e0c102e0ad23165e8b7157669819302ddcb3c68226`	OK
`"hello"`	`2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824`	`3a11bd11262fcee01e4a184372643bdbe4ba65651b730b0b8fe49c7648866d52c60847db2400ef72c859e2b0366e470fae3cf0891313b4`	OK
`"padded test 1"`	`146f48dd2a3324d46feb7ad2069e05ddb9530be67f8f823f9d1c2ed3ea4f1d19`	`e2609a7b6efe43cb60db80500104189d140405a2f6b372f4e661055713c878065bc57295da59a70765407c34475c60142000cc934c6892`	OK
`"padded test 2"`	`4c51f9dde95fa3ec5545b46ca0f2d8968b2399f81f208e07304f4b00117c02df`	`b3747f89662c16d1327d8192f8dd854303bf399c7ab5dcb422fe7a287485ddc9b35c9445ac693d361ad35ee0c33f09b9e745379347451c`	OK
`"padded test 3"`	`cb9ace26d0eee1034f838de327a72598832bfa4fa1644493ad4ef004a5a2dab9`	`2a0a537e5f36b42409261161e56b241e4a816492cee5fd6897085892258dfab11ae9f922586a475b0dd2eb475d977bedaf388a94db584e`	OK
`"padded test 4"`	`90b183e68a4966c5c268807fe3f49a4f4cf890f9f25b00bb1f836552496192f4`	`312115c5023df0d90166fbb70630c77a072454beac6c71baf561dbd7644d889a339b5a1adf224599d226c509fb513fcc0e517463823afe`	OK
`"padded test 5"`	`286d96b78dc5b71f90b61e6374de3ffc82939941955e7b1c5d00fd9af345e80a`	`ac7dc4c258ea88f0d33b5adf6228fbb80f5a5e97e3f4b87452358067a178da46342609beddd43edd7b675a4c4bacfa772e3c0ed0655893`	OK
`"padded test 6"`	`efa23b479b7518cb4117dfdf041a694b46441fa5796e6d4b8a7443a724a0d8f1`	`56b98be1c35ba9d631f39ef2183cae680c4f348dbf11782f3e26fa17f5d09f480a3a03fe2ec02b3a0b5b93a42c274a830b78b19aa2b732`	OK
`"padded test 7"`	`2f507443a19f9f39300d5905b02aa1cd77c744f0424c684f75a5b2cc4a0516a8`	`40c32d88f0d6237161926436e6ff445c06cf067f56a89408e54f57482c0d109aca8bcdb09d72c5253118cd7f0e101ccc5a68e0713a6f30`	OK
`"padded test 8"`	`1fad9d7e79a132660d8536e6b151a4cef494bdfdc2962e0f7bae20acd8aed5c8`	`0b5845af8f13991a7f034a996d7cf1d4f73281307f02fde1cf59c8c86a858e24769d720c8327317e3a15f64bd78caf7cdfd85b0eb7ec12`	OK
`"padded test 9"`	`849564c85e22852e6af2c6ec60bd891de79298bbc8d5106a221bca536a908d1f`	`3917b784b65b516ef8c6a7c8b5efa09eb23d18265633d06304b438f75939ee5066bee8cf9e191247ada9185debc9c3005723e901ef6bf4`	OK
`"padded test 10"`	`91e12eb79793591d6773d8c3b5090f07a69742c7031bbd514273c5de1c241c87`	`64a7c6d93f0cf5272ef808a2fd2fa357c6e7f285ee9acb7fbafc26ae75df1f9e69231cc929e0a799b9aff6f0adfacf5f46790ff30d188b`	OK
`"padded test 11"`	`7b66f998c1c60bdd65744e43b69190481502cee718245876fc2c535f4b7cf5a8`	`95ae4eac83a9011e5566e562f14507b0dd53f8f6391a4b89718a66d9ee5a9b02366e31a8e7345cf14b70b5d82600456857a68161b2ee3b`	OK
`"hand-off cross-check 2026-09-29"`	`d374822f21eb0d4dbc809ae0443c2c6c66f8ce5d07c8779bcc5fc7849f44e3c2`	`700821b0948103e6e0ee472f6e7fac2a44139bc265ad13f422dc966e00e3c17e1f472e92b1769a7b9199a2a704fa97c927e01939f97333`	OK

Table 8.3: Complete 19-step single-block FIPS 180-4 padded benchmark suite (all 14 public-label targets, full 55-byte `message_hex`, $L = 440$ bits, generated by our soft-shaping 19-step padded solver `src/pre19.c` with $D = \mathrm{popcount}(a_5 \oplus a_6) \leq 3$ and $e_9 = e_{10} =$ `0xffffffff`).

Target Label (`H* = SHA-256(label)`)	Complete 19-Step Padded 55-Byte Preimage `message_hex` (110 hex chars; pad suffix `8000000000000001b8`)	Verified
`"abc"`	`8ca254f8d04dd3fb04efaad6b1867588ffe833de49a144fedbf1839422892dc9f5a9158d8fc063290a14a9a6c55dfa245642a0247ec45a`	OK
`"hello"`	`ac055aaa83e1e51a5d74102b7739b7b6e019e31100cd525baf09c642d4b59b8fc4eb8d481df111aee2616325b336b827190103c33584c5`	OK
`"padded test 1"`	`a9cfb3f277e9e66e5a86ea3ff1dac69d3327b726deb9420583f150597ba70fa19cfc18400c1222c58d20911e6162081ebe084915634c3a`	OK
`"padded test 2"`	`c13089d977eea9843460eb14fff02051a5cf91b4ffedf39711c2ec81862939f8adcc0dc25fb391e5c84ed20f8ef02e4a205578ea45e9fd`	OK
`"padded test 3"`	`e24cf94c817e32304c4162886506bdfa5d2766398d36171235b6e3fde064535456052c3022ef488175bbbd6b81085c86401df4e0dae715`	OK
`"padded test 4"`	`3b605fe04bdf7b05dbcae817e1a77d8e7e9ad63db1fe8f5d972828874695dde35bcf2deb7166e78132c9ac7ee9f72d07190b83c398a2ed`	OK
`"padded test 5"`	`502eb30d876c48eaee8a13c08e4e15b5ac74689dfdf5e60cb2fd044d4bc8eaae640c92cf6236d908717a5842304f70d47fc4dddbe6d899`	OK
`"padded test 6"`	`782f45dda1ab000aace616e7c3076339fe8938a6f0a35fc6ff0e2739c3194c08494e5b1a4af0b10e69d87a02352ddd71034511dba58e90`	OK
`"padded test 7"`	`fb676bdeabfc0143be3d2b4c8350083e39b4aedcdfbf2b87c10c23786a901dc14ffc2879516522cbc86235d1a7bf135373355d0cf71888`	OK
`"padded test 8"`	`a1edbba02a9f54cf4083f7fc27156523f3acd5f52339e453c950d9ed841486dff2264d752bea2d27440984bd612c77f69b4479b63cde7f`	OK
`"padded test 9"`	`0736a2716d8ee60644b77f1e85c91f230ed82d7f242ae673baeb3049b858b159e8d90ba74901bda34b0536a82b222f425f9533e25c727c`	OK
`"padded test 10"`	`a80e3ae94cb0c81f3e591bfd4453eba61c2148ab7d32f1e524c220134d9be8e12ee2beebfa02fb6bc994272b720842d68709f1e0023b79`	OK
`"padded test 11"`	`23ae3771d98d69c6925d4ca4b8059f84404eea570db3d014c022af5649565a9a19a0e610fdb9258ce144893cee5d25f260f11614b36253`	OK
`"hand-off cross-check 2026-09-29"`	`6e77d54453883692c4e2fbf20d290ce36ee98f42748f7c618cd07a0d19b36b5a43c25fa3b008d57af11dbc2dabc843dae6117a0f867286`	OK

- **Demonstrated 20-Step Unpadded Compression Preimage (`deadbeef...` , Zero-a_3 Bridge `solve20_zero_a3.c` , plus all 12 `[comp]` targets `"unpadded test 1"` .. `"unpadded test 11"` and `"hand-off cross-check 2026-09-29"` in `witnesses_20step.txt`):**

```
W[ 0.. 7] = 4334d956 6eccece1 6727a1b2 d85811af 890c5399 79018a3b d346b94d 5f26009b
W[ 8..15] = 7562d311 0bfd80d5 1f8c38f4 16d8315d e0353aa2 6adf4bd2 a2bf14a2 801ae8f7
```

---

9. Experimental Ablation, Scope, and Structural Complexity Bounds

9.1 Algorithmic Ablation Across CPU and GPU

Table 9.1 summarizes the algorithmic ablation of our solver on genuine, unmutated right-hand contexts from soft-shaped CSR tables to the full Carry-Sudoku / Slot-Sudoku architecture (see Table 1.1 in §1.1 for the comparison against prior reduced-step SHA-256 preimage attacks). For CPU configurations, we report `CPU-ref` per `sol19` against our measured same-host 10-core scalar CPU reference baseline (1.5125×10^8 SHA20-eq/s on arbitrary 20-step message blocks). Supplementary GPU telemetry on 1× NVIDIA L4 (`solve20_padded_cuda.cu` , `solve20_padded_cuda_prod.cu`) is reported in direct GPU wall-clock time on genuine contexts.

Table 9.1: Algorithmic ablation of 19-step bridge candidate (`sol19`) and 20-step padded preimage throughput on genuine unmutated contexts.

Algorithmic Configuration	Q_0^{-1} Table & Memory	Right-Context Rate ($D = 0$)	19-Step Bridge Rate (`sol19/s`)	`CPU-ref` Evals per `sol19`	Expected 20-Step Time (1× Device)	Expected Time (8× Devices)
Chosen-Target 19-Step Soft Shaping (`src/pre19.c`, 1 core)	2.0625 GiB CSR (2^{29} entries, $D \le 3$)	Soft ($\approx 2^{26.7–26.9}$ trials/ctx)	Chosen H_{19}^* search	— ($2^{20.8–26.8}$ chains; $2^{32.51–32.87}$ evals)	**`1.2–86.4 s` (19-step, med. `19.45 s`)**	—
Exact Shaping + Slot-Sudoku CPU (`solve20_padded_fast.c`, 16 threads)	6.00 GiB 2-Tier 4-Slot EDF (`[E30]` – `[E31]`)	266.8 ctx/s	6,242.8 sol19/s	24,228 ($2^{14.56}$)	**`191.1 node-h`** ($2^{46.56}$ CPU-ref)	**`23.9 node-h`** (8× CPU)
Slot-Sudoku + 5-Step $\mathbb{F}_2$ Inverse GPU (*supp.*, `[E29]`, 1× L4)	6.00 GiB 2-Tier EDF ($B = 8$ sector)	7,580 ctx/s	53,847.5–73,560.5 sol19/s	— (GPU slice telemetry)	16.22–22.16 GPU-h	2.03–2.77 GPU-h
Direct 2-Way + Genuine Exact-T_{16} Twin GPU (*supp.*, `[E32]`, 1× L4)	**16.00 GiB Direct 2-Way (`twin2` / `single4`)**	29,310–114,507 ctx/s	132,888.7 sol19/s (genuine twins)	— (GPU slice telemetry)	**`8.98 GPU-h`** (genuine twins)	**`1.12 GPU-h`** (8× L4)

(Note: The experimental `--bench-c2` through `--bench-c10` harnesses in `solve20_padded_cuda.cu` [Cycles 2, 4, and 7–10, including `run_bench_cycle2` which recorded 180,831.9 sol19/s / 6.60 GPU-h by forcing 16 contexts to share a single T_{16}] are synthetic architectural microbenchmarks; we exclude those figures from Table 9.1 and report only genuine-context rates.)

9.2 Scope, Terminology, and Limitations: Subspace-Restricted Preimages, Degree-of-Freedom Budget, Exact-T_{16} Twin Capacity, and Construction-Specific Bottleneck Analysis

To situate our results accurately within the SHA-256 cryptanalysis literature, we distinguish three notions of restriction in reduced-step preimage attacks:

1. **Unpadded Compression Function vs. Structured-Target Padded vs. Arbitrary-Target Full FIPS 180-4 Padded Hash Function:** Prior practical 18-, 19-, and 20-step results [6, 7] and the unpadded solvers in [9] restrict their claims to the **unpadded compression function** ($W_0, \dots, W_{15} \in \{0,1\}^{512}$ unconstrained); Zaikin [7] specifies that Merkle–Damgård padding is omitted while keeping the standard IV and feed-forward. Basra [9] additionally demonstrated a 55-byte 20-step padded preimage for a structured target (`padded_r20/README.md` in [9], dated 17 October 2026). Our 19- and 20-step solvers (`src/pre19.c`, `solve20_padded_fast.c`, `solve20_padded_cuda.cu`, `solve20_padded_cuda_prod.cu`) satisfy all 72 bits of single-block FIPS 180-4 padding [1] ($W_{13} \bmod 256 = \texttt{0x80}$, $W_{14} = 0$, $W_{15} = 440$) for **arbitrary externally prescribed 256-bit targets**, producing valid 55-byte (and 119-byte two-block) messages across all 16 padded 20-step and 16 padded 19-step targets in §8.

2. **Subspace-Restricted Preimages, Exact-T_{16} Twin Capacity, and the Degree-of-Freedom Budget:** Like all splice-and-cut MitM and biclique attacks on padded SHA-256 [2, 3, 4, 8, 10], which fix message lengths in (W_{14}, W_{15}) and pin internal state registers ("neutral words" / "initial structures") to decouple forward and backward chunks, our solver restricts its search to a structured **55-byte single-block internal-state manifold** defined by `[E8]`, `[E11]`, `[E13]`, and `[E22]`. Because a 55-byte message contains 440 free message bits against a 256-bit target digest H^*, the unconstrained 55-byte padded message space holds $2^{440-256} = 2^{184}$ expected preimages per 256-bit target for **both** the 19-step and 20-step hash functions. When searching for a 20-step preimage, the solver enumerates 19-step bridge candidates (`sol19`) that satisfy Steps 1–19 of the 20-step target (leaving the 32-bit terminal equation $\sigma_0(W_4) + W_3 = T_{19}$ open), so under a heuristic 2^{-32} terminal survival model the expected number of `sol19` bridge candidates for a given 20-step target is 2^{32} times the expected number of 20-step preimages inside the same restricted family. Table 9.3 tracks these model expectations across the 20-step manifold constraints.

Table 9.3: Distributional degree-of-freedom budget across the 20-step 55-byte single-block padded manifold.

Constraint / Algebraic Pinning	Equation	Bits Spent	Model Expected 20-Step Preimages per Target	Model Expected `sol19` Bridge Candidates per 20-Step Target
Full 55-byte padded message space (W_{13} pad, $W_{14} = 0, W_{15} = 440$)	`[E1]`	72.0 bits (of 512)	$2^{184.00}$	$2^{216.00}$
Selector saturation ($e_9 = e_{10} = \texttt{0xffffffff}$) [9]	`[E8]`	64.0 bits	$2^{120.00}$	$2^{152.00}$
Exact context shaping ($a_5 = a_6 \iff D_{\text{shape}} = 0$)	`[E11]`	32.0 bits	$2^{88.00}$	$2^{120.00}$
Majority absorption ($\mathrm{Maj}(a_5, a_4, a_3) = a_4$ on $w \le 14$)	`[E13]`	13.40 bits ($13.28 + 0.12$)	$2^{74.60}$	$2^{106.60}$

Constraint / Algebraic Pinning	Equation	Bits Spent	Model Expected 20-Step Preimages per Target	Model Expected `sol19` Bridge Candidates per 20-Step Target
Right-context Ch-annihilation ($e_{15} =$ `0xffffffff`, $a_{10} = a_{11}$)	`[E22]`	64.0 bits	$2^{10.60} \approx 1{,}552$ ($2^{8.69} \approx 413$ in 2-way table)	$2^{42.60}$ ($2^{40.69}$ in 2-way table)

- **Carry-Sudoku Context Universe ($U \approx 1.68 \times 10^7$) vs. Exact-$T_{16}$ Twin Capacity (`single4` Mode):** Across all 2^{32} values of a_{12} for a given 20-step target H_{20}^*, the complete 7-stage Carry-Sudoku cascade (`[E22]` – `[E25]`, Algorithm 1) produces an estimated universe of $U \approx 1.68 \times 10^7 \approx 2^{24.00}$ valid right-hand contexts (or $U_{\text{filt}} \approx 1.014 \times 10^7 \approx 2^{23.27}$ when the retired heuristic $\Sigma_0(a_9)$ window check `[E26]` was enabled). In our 2-way table configuration (`[E32]`), each context yields an empirical average of $\approx 1.06 \times 10^5 \approx 2^{16.69}$ 19-step bridge candidates (`sol19`) across $a_1 \in [0, 2^{32})$, so the estimated universe U yields $\approx 2^{24.00} \times 2^{16.69} = 2^{40.69}$ reachable `sol19` bridge candidates ($\approx 413 = 2^{8.69}$ expected 20-step padded preimages per target under the 2^{-32} terminal model out of the $2^{10.60} \approx 1{,}552$ unfiltered manifold expectation, or $\approx 250 = 2^{7.97}$ when restricted to U_{filt}). Under a uniform birthday model over $U \approx 1.68 \times 10^7$ contexts, the expected number of natural Exact-T_{16} twin pairs ($T_{16}^{(i)} = T_{16}^{(j)}$) is $N_{\text{twins}} \approx U^2/2^{33} \approx 32{,}890$ pairs ($65{,}780$ contexts), yielding $\approx 65{,}780 \times 1.06 \times 10^5 \approx 1.62 \times 2^{32}$ `sol19` candidates ($1 - e^{-1.62} \approx 80.3\%$ Poisson success probability from natural twins alone; if restricted to U_{filt}, $N_{\text{twins}} \approx 11{,}970$ pairs yielding $\approx 0.59 \times 2^{32}$ `sol19` candidates or 44.6% Poisson success). Across our 16 padded 20-step targets, `twin2` (`sweep_exact16_cuda_kernel`) solved 12 targets directly; when natural twin pairs in a sampled pool are exhausted, the non-twin 4-way reservoir kernels (`single4` via `sweep_20_warp_l3_prod_kernel` in `solve20_padded_cuda_prod.cu` and `sweep_exact16_cuda_kernel` in `solve20_padded_cuda.cu`) sample directly from the full context reservoir, solving the remaining 4 targets (16/16 total).
- **Binary 55-Byte Output Format vs. Two-Block Chosen-ASCII Prefix (119 Bytes):** In a single 55-byte block, $W_0, \ldots, W_{10}$ and the upper 24 bits of W_{13} are determined algebraically by the state trajectory. As demonstrated in §8.3, a **119-byte two-block message** ($W_{15}^{(2)} = 952 =$ `0x3b8`) allows the first 64-byte block B_1 to be arbitrary ASCII text while Block 2 is solved from chaining state $\mathrm{IV}' = \mathrm{Compress}_{20}(\mathrm{IV}, B_1)$ at identical complexity.

3. **Construction-Specific Bottleneck Analysis and 15 Explored Sub-2^{32} Directions:** Within our decoupled Left/Right bridge architecture (`[E8]` – `[E14]`), each step beyond Step 19 adds one schedule relation $W_t = \sigma_1(W_{t-2}) + W_{t-7} + \sigma_0(W_{t-15}) + W_{t-16}$. By `[E37]`, σ_0 is an $\mathbb{F}_2$-linear bijection on $(\mathbb{F}_2)^{32}$ ($\mathrm{rank}_{\mathbb{F}_2}(\sigma_0) = 32$), so for any fixed Right context and Level-3 bridge leaf (W_1, W_2, W_3), the terminal relation $\sigma_0(W_4) = T_{19} - W_3 \pmod{2^{32}}$ uniquely determines $W_4 = \sigma_0^{-1}(T_{19} - W_3 \pmod{2^{32}})$, acting as an exact 32-bit point filter against the value of W_4 determined by (a_1, a_2, a_3, a_4) (though $\det_{\mathbb{F}_2}(\sigma_0) = 1$ alone does not prove statistical uniformity of T_{19} across the bridge stream). To investigate whether an algebraic, differential, polynomial, cross-round, or multi-block modification could bypass this 32-bit terminal filter within our framework, we formulated and evaluated **15 candidate sub-2^{32} attack directions** (`test_all_12_theories.c`, Table 9.4). We emphasize that Table 9.4 is a taxonomy of construction-specific negative results and surrogate empirical checks rather than a universal impossibility proof over all conceivable algorithms.

Table 9.4: Taxonomy of 15 construction-specific candidate sub-2^{32} directions evaluated against the 20-step terminal filter.

#	Candidate Sub-2^{32} Direction Evaluated	Mathematical Mechanism	Empirical / Algebraic Result	Observed Bottleneck
1	Rightward Spine Shift ($e_{10} = e_{11} =$ `0xffffffff`)	Shift bridge wall from a_5 to a_6 to gain 5 Left variables	W_{16} requires $W_9(a_5, e_9)$ at Level 1 before a_5 is known (`W16 <-> W9` Causality Law)	Circular dependency ($\geq 2^{32}$)
2	Direct $\mathbb{F}_2^{32}$ Terminal Inversion (`[E37]`)	Invert $\sigma_0(W_4) = T_{19} - W_3$ directly via $\det_{\mathbb{F}_2}(\sigma_0) = 1$	Uniquely locks W_4, which overdetermines a_4 against Level-3 $Q_0(W_3) = t_{18}$	32-bit filter ($\approx 2^{32}$)
3	Backward 4-Level Tree ($W_4 \to W_3 \to W_2 \to W_1$)	Start from T_{19} and walk backward to a_1	W_4 requires (e_4, e_3, e_2, e_1), which depend non-triangularly on (a_4, a_3, a_2, a_1)	Non-causal ($\geq 2^{32}$)
4	Cross-Context Meet-in-the-Middle (T_{19} Multiplexing)	Reuse one Left chain $(a_1..a_4)$ across M Right contexts	Left chain depends on the 96-bit triple (T_{16}, T_{17}, T_{18}); collision requires $\approx 2^{48}$ contexts	Birthday wall ($2^{48} \gg 2^{32}$)
5	LSB Carry-Chain Early Pruning on $\sigma_0(W_4) + W_3$	Prune a_4 bit-by-bit using LSBs of T_{19}	$\sigma_0(W_4)$ mixes MSB bits of W_4 into bit 0 via $\mathrm{ROR}(W_4, 7) \oplus \mathrm{ROR}(W_4, 18)$	Full 32-bit mixing
6	Differential / Boomerang Trail on Surrogate 3-Level Map	Flip bit i in surrogate 3-level Q_0 chain (`test_theory_14`)	Measured across all 32 bit flips in surrogate model: $\approx 16.0/32$ bits avalanche	No observed differential shortcut
7	Linear / Walsh-Hadamard Approximation	Correlate linear masks $\alpha \cdot a_1 \oplus \beta \cdot T_{19}$	Alternating modular addition and $\mathbb{F}_2$-linear Q_0^{-1} across 3 levels rapidly damps linear trails	No observed linear shortcut
8	$\mathbb{F}_2$ / ANF Polynomial Degree Reduction (Gröbner / SAT)	Express $T_{19}(a_1)$ as low-degree boolean polynomials	3-level modular addition + Q_0^{-1} carry chains reach high algebraic degree	High ANF degree

#	Candidate Sub-2^{32} Direction Evaluated	Mathematical Mechanism	Empirical / Algebraic Result	Observed Bottleneck
9	Invariant Subspace / Fixed-Point Manifold of Q_0^{-1}	Restrict W_1, W_2, W_3 to σ_0-invariant subspaces	Affine shifts T_{16}, T_{17}, T_{18} break $\mathbb{F}_2$-subspace invariance across levels	No invariant subspace found
10	2-Way Bi-Directional Bridge inside Single Block ($a_1 \leftrightarrow a_4$)	Meet at a_2 or a_3 from both ends of Left chain	e_3 and e_4 couple (a_4, a_3, a_2, a_1) across 4 steps simultaneously; memory $\geq 2^{32}$	$\approx 2^{32}$ time-memory
11	Soft-Shaping Relaxation ($D_{\text{shape}} = a_5 \oplus a_6 \neq 0$)	Allow $D_{\text{shape}} \neq 0$ to let a_4 steer W_{11}, W_{12} into T_{19}	Every non-zero bit in D_{shape} halves Level-3 survival, offsetting T_{19} freedom	Net gain $\leq 1\times$
12	Unfrozen $e_{15} \neq$ `0xffffffff` / $a_{10} \neq a_{11}$ Right Coupling	Vary e_{15} or $a_{10} - a_{11}$ so multiple contexts share (T_{16}, T_{17}, T_{18})	Enlarges right-context pool to $\approx 2^{74.60}$ model solutions, but each context still has its own (T_{16}, T_{17}, T_{18})	$\approx 2^{32}$ `sol19` per hit
13	**4-Round Crossed Carry-Sudoku Spine ($a_2 = a_3 = a_4 = a_5$, `[E45]`)**	Pin $a_2 = a_3 = a_4 = a_5$ so W_1, W_2, W_3, W_4 are $O(1)$ with 0 Q_0^{-1} lookups	Specialized spine kernel runs at 9.62G evals/s on L4 GPU, but spends 96 Left bits, turning T_{16}, T_{17}, T_{18} into a 96-bit Right filter	Conservation of DoF
14	**5-Step Crossed Spine ($a_1 = a_2 = a_3 = a_4 = a_5$, `[E45]`)**	Pin all 5 Left words so $e_3 = a_{-1} - \Sigma_0(a_5)$	Verified algebraically; spends all 128 Left bits, requiring 2^{128} Right contexts	Conservation of DoF
15	**Two-Block Feed-Forward Coupling ($M = B_1 \parallel B_2$, 119 Bytes)**	Use free 512-bit Block 1 (B_1) to steer chaining state $V = H^{(1)}$ into Block 2	By Davies–Meyer feed-forward, $H_{20}^* = V + S_{20}^{(2)} \pmod{2^{32}}$: fixing $S_{20}^{(2)}$ uniquely locks all 256 bits of $V = H_{20}^* - S_{20}^{(2)}$	**Two-Block Locking**

- **Analysis of Direction 15 (Two-Block Feed-Forward Locking):** In a two-block 119-byte padded message $M = B_1 \parallel B_2$, Block 1 ($B_1 \in \{0,1\}^{512}$) maps $\text{IV} \mapsto V = \text{Compress}_{20}(\text{IV}, B_1)$, and Block 2 maps $V \mapsto S_{20}^{(2)}$ with Davies–Meyer feed-forward $H_{20}^* = V + S_{20}^{(2)} \pmod{2^{32}} \iff S_{20}^{(2)} = H_{20}^* - V \pmod{2^{32}}$. Because V enters Block 2 at **both** boundaries simultaneously (Step 0 as V and Step 20 as $H_{20}^* - V$), choosing B_1 first fixes both boundaries and reduces Block 2 to a single-block 20-step padded problem, while choosing Block 2's terminal state $S_{20}^{(2)}$ first uniquely locks all 256 bits of $V = H_{20}^* - S_{20}^{(2)}$, requiring a full 256-bit 20-step compression preimage on Block 1.
- **Resulting Construction-Specific Scaling:** While the unconstrained 55-byte padded message space (2^{440} messages) has $2^{440-256} = 2^{184}$ expected preimages per 256-bit digest at any step count R, within our fixed-wall Left/Right bridge construction (`[E8]` + `[E11]` + `[E13]`), **19-step bridge candidates (`sol19`)** are generated in $O(1)$ amortized work ($24{,}228 \approx 2^{14.56}$ CPU-ref evaluations per `sol19` , or 1.2–86.4 s for a standalone chosen-target 19-step padded preimage in `src/pre19.c`), whereas **20-step padded preimages** require testing the terminal schedule relation $\sigma_0(W_4) + W_3 \equiv T_{19} \pmod{2^{32}}$, costing an expected 2^{32} `sol19` candidates under a uniform terminal residual model (191.1 node-h / $2^{46.56}$ CPU-ref on 1× CPU node, or 8.98–22.16 GPU-h in supplementary 1× L4 GPU slice telemetry). Extending this same fixed-wall architecture beyond 20 steps shifts the terminal state boundary $(a_{R-7}, \ldots, a_R)$ rightward and introduces additional schedule equations $W_{19}, \ldots, W_{R-1}$ that each impose an additional 32-bit constraint unless new algebraic degrees of freedom are opened.

Disclosure & Attribution

The selector-saturation and context-shaping principle ($e_9 = e_{10} =$ `0xffffffff`, $a_5 \approx a_6$), together with the observation that the FIPS 180-4 padding words (W_{13}, W_{14}, W_{15}) can be absorbed backward into (a_7, a_6, a_5) at the cost of three context words and a 20-step padded witness for a structured target, is due to K. S. Basra [9] (`README.md` , `padded_r19/README.md` , `padded_r20/README.md` , and `paper/sha256_r19_preimage.pdf` in `https://github.com/kamb-code/sha256-r19-preimage`), whose manuscripts note substantial assistance from AI tools. Both `srw/sha256-19step-preimages-with-padding` and the algebraic discoveries (closed-form $O(1)$ W_{13} padding pivot inversion, Theorem 1 / Carry-Sudoku, Theorem 2 / closed-form $\mathbb{F}_2$ upper-half reconstruction, Slot-Sudoku tables, and the Zero-a_3 bi-directional bridge), C/CUDA solvers, and preimage witnesses in this work were discovered and developed using **Google DeepMind Gemini 4 Argon**.

References

1. **[1]** National Institute of Standards and Technology: *Secure Hash Standard (SHS)*. FIPS PUB 180-4, U.S. Department of Commerce (2015).
2. **[2]** Isobe, T., Shibutani, K.: *Preimage Attacks on Reduced Tiger and SHA-2*. In: Dunkelman, O. (ed.) Fast Software Encryption – FSE 2009, LNCS, vol. 5665, pp. 139–155. Springer (2009).
3. **[3]** Aoki, K., Guo, J., Matusiewicz, K., Sasaki, Y., Wang, L.: *Preimages for Step-Reduced SHA-2*. In: Matsui, M. (ed.) Advances in Cryptology – ASIACRYPT 2009, LNCS, vol. 5912, pp. 578–597. Springer (2009).
4. **[4]** Khovratovich, D., Rechberger, C., Savelieva, A.: *Bicliques for Preimages: Attacks on Skein-512 and the SHA-2 Family*. In: Canteaut, A. (ed.) Fast Software Encryption – FSE 2012, LNCS, vol. 7549, pp. 244–263. Springer (2012).

5. **[5]** Homsirikamol, E., Morawiecki, P., Rogawski, M., Srebrny, M.: *Security Margin Evaluation of SHA-3 Contest Finalists Through SAT-Based Attacks*. In: Cortesi, A. et al. (eds.) CISIM 2012, LNCS, vol. 7564, pp. 56–67. Springer (2012) (16-round SHA-256 SAT evaluation in ~1 h, Table 1).
6. **[6]** Davydov, V. V., Pikhtovnikov, M. D., Kiryanova, A. P., Zaikin, O. S.: *Cryptographic Strength Analysis of the SHA-256 Hash Function Using the SAT Approach*. Scientific and Technical Journal of Information Technologies, Mechanics and Optics, vol. 25(3), pp. 428–437 (2025). DOI: `10.17586/2226-1494-2025-25-3-428-437`.
7. **[7]** Zaikin, O.: *Preimage Attacks on Round-Reduced MD5, SHA-1, and SHA-256 Using Parameterized SAT Solver*. Constraints (published 23 Jan 2026). DOI: `10.1007/s10601-025-09383-0` (19-step unpadded all-ones target in 18 h 33 min on 192 CPU cores, §5 / Table 2).
8. **[8]** Guo, J., Li, H., Liu, M., Wang, S., Zhang, T.: *Dual-Syncopation Meet-in-the-Middle Attacks: New Results on SHA-2 and MD5*. In: Advances in Cryptology – EUROCRYPT 2026. Springer (2026); IACR ePrint 2026/353.
9. **[9]** Basra, K. S.: *Oracle-Free Preimage Attack on 19-Round Reduced SHA-256*, *The 20-Round Barrier for Table-Based Preimage Attacks on SHA-256*, and `README.md`, `padded_r19/README.md`, `padded_r20/README.md` (structured-target 20-round padded witness, 17 Oct 2026). GitHub repository and manuscripts, `https://github.com/kamb-code/sha256-r19-preimage` (Aug–Oct 2026).
10. **[10]** Sasaki, Y., Wang, L., Aoki, K.: *Preimage Attacks on 41-Step SHA-256 and 46-Step SHA-512*. IACR Cryptology ePrint Archive, Report 2009/479 (2009).
11. **[11]** Mendel, F., Nad, T., Schläffer, M.: *Improving Local Collisions: New Attacks on Reduced SHA-256*. In: Johansson, T., Nguyen, P. Q. (eds.) Advances in Cryptology – EUROCRYPT 2013, LNCS, vol. 7881, pp. 262–278. Springer (2013).
12. **[12]** Mendel, F., Nad, T., Schläffer, M.: *Finding SHA-2 Characteristics: Searching through a Minefield of Contradictions*. In: Lee, D. H., Wang, X. (eds.) Advances in Cryptology – ASIACRYPT 2011, LNCS, vol. 7073, pp. 288–307. Springer (2011).
13. **[13]** De Cannière, C., Rechberger, C.: *Finding SHA-1 Characteristics: General Results and Applications*. In: Lai, X., Chen, K. (eds.) Advances in Cryptology – ASIACRYPT 2006, LNCS, vol. 4284, pp. 1–20. Springer (2006) (generalized characteristic framework introduced for SHA-1 and adapted to SHA-2 in [11, 12]).
14. **[14]** Li, Y., Liu, F., Wang, G.: *New Records in Collision Attacks on SHA-2*. In: Joye, M., Leander, G. (eds.) Advances in Cryptology – EUROCRYPT 2024, LNCS, vol. 14651, pp. 158–186. Springer (2024); IACR ePrint 2024/349 (39-step SHA-256 semi-free-start and 40-step SHA-224 free-start collisions).
15. **[15]** Dobraunig, C., Eichlseder, M., Mendel, F.: *Analysis of SHA-512/224 and SHA-512/256*. In: Iwata, T., Cheon, J. H. (eds.) Advances in Cryptology – ASIACRYPT 2015, LNCS, vol. 9453, pp. 612–630. Springer (2015).
16. **[16]** McDair: *17-Step Reduced SHA-256 Preimage Challenge Witnesses*. Cryptography Mailing List (`metzdowd.com`, `2024-April/001334.html`), Mar–Apr 2024 (archived mailing-list posts of three 17-step messages without a published attack method).
17. **[17]** Stevens, M., Bursztein, E., Karpman, P., Albertini, A., Markov, Y.: *The First Collision for Full SHA-1*. In: Katz, J., Shacham, H. (eds.) Advances in Cryptology – CRYPTO 2017, LNCS, vol. 10401, pp. 570–596. Springer (2017).
18. **[18]** Leurent, G., Peyrin, T.: *SHA-1 is a Shambles: First Chosen-Prefix Collision on SHA-1 and Application to the PGP Web of Trust*. In: 29th USENIX Security Symposium (USENIX Security 2020), pp. 1839–1856 (2020).